

Convolutional Neural Network

by

Nada Fitrieyatul Hikmah

Simulasi NN

- [A Neural Network Playground \(tensorflow.org\)](https://tfplayground.tensorflow.org)

Spiral Dataset → using architecture “Think Wide”



Epoch
000,474

Learning rate
0.01

Activation
ReLU

Regularization
L2

Regularization rate
0.003

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

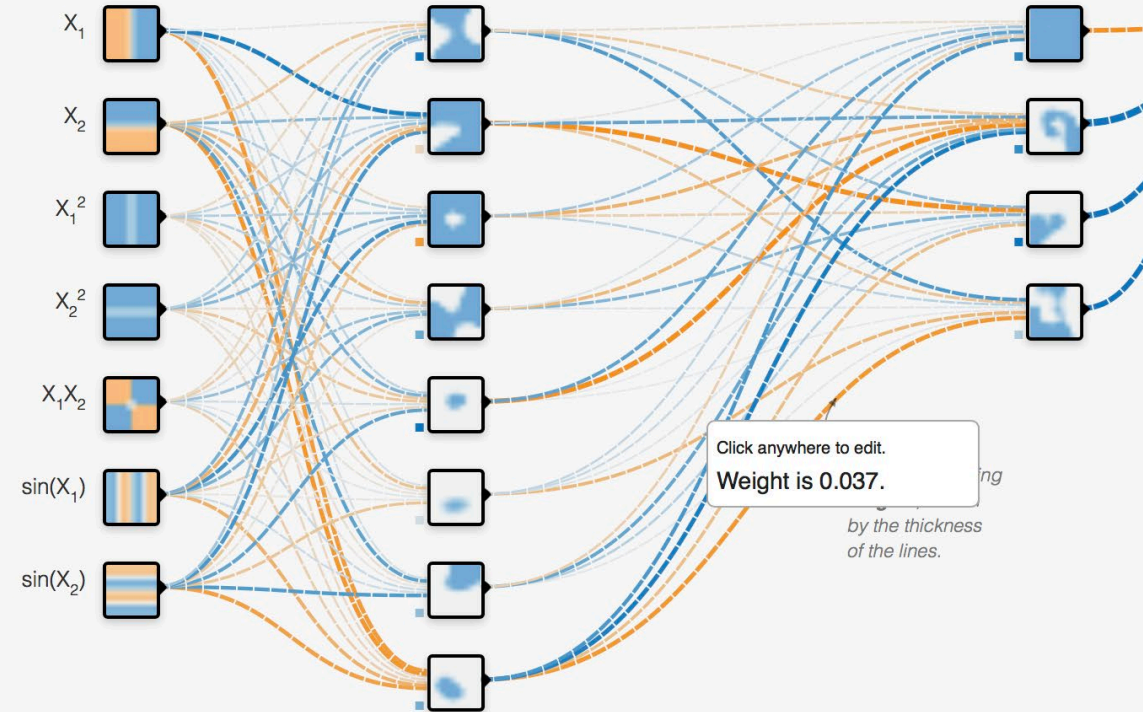
Which properties do you want to feed in?

X_1
 X_2
 X_1^2
 X_2^2
 $X_1 X_2$
 $\sin(X_1)$
 $\sin(X_2)$

2 HIDDEN LAYERS

8 neurons

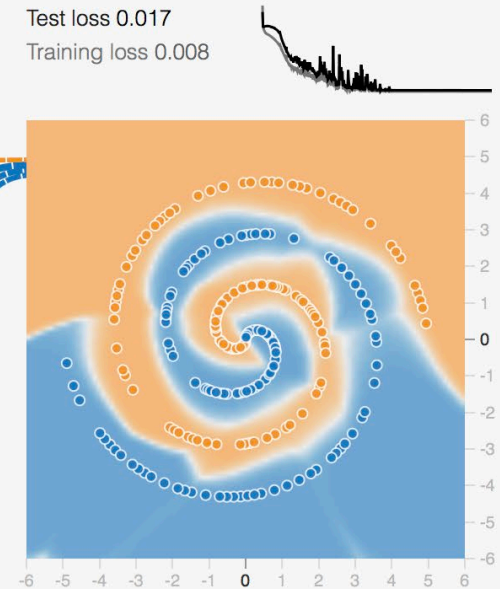
4 neurons



This is the output from one neuron. Hover to see it

OUTPUT

Test loss 0.017
Training loss 0.008



Colors shows data, neuron and weight values.

☐ Show test data

☐ Discretize output

Spiral Dataset → using architecture “Think Deep”

⏮ ⏪ ⏩ ⏭

Epoch: 000,610

Learning rate: 0.03

Activation: ReLU

Regularization: L2

Regularization rate: 0.003

Problem type: Classification

DATA

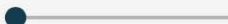
Which dataset do you want to use?



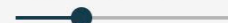
Ratio of training to test data: 50%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?

X_1



X_2



X_1^2



X_2^2



$X_1 X_2$



$\sin(X_1)$



$\sin(X_2)$



+ - 3 HIDDEN LAYERS

+ -

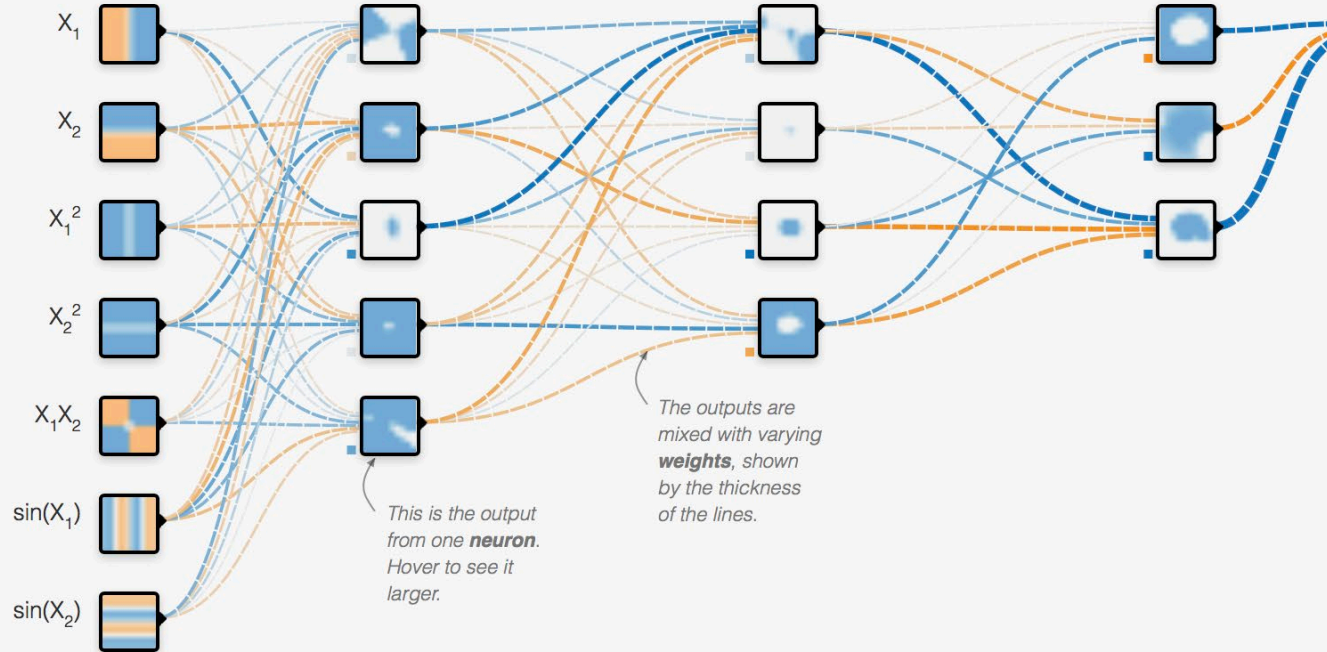
5 neurons

+ -

4 neurons

+ -

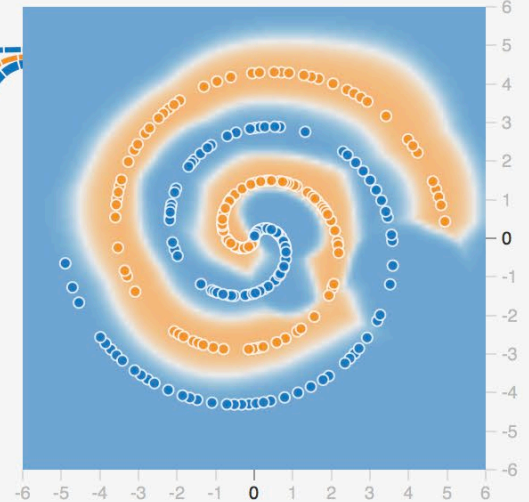
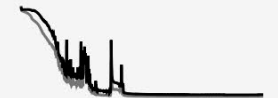
3 neurons



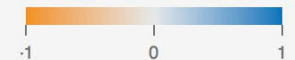
OUTPUT

Test loss 0.017

Training loss 0.006



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

CPU and GPU

- RAM (memory) → tempat penyimpanan sementara → cepat diakses, sehingga butuh durasi waktu yang singkat untuk mengambil data.
- CPU (Central Processing Unit) → didesain untuk komputasi sekuensial/serial → untuk komputasi, processing.
- GPU (Graphic Processing Unit) → didesain untuk komputasi yang parallel → ada memorynya sendiri.

PyTorch

Nada F. H.

Installation

- Open : [Start Locally | PyTorch](#)

PyTorch Build	Stable (1.11.0)		Preview (Nightly)	LTS (1.8.2)
Your OS	Linux		Mac	Windows
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 10.2	CUDA 11.3	ROCm 4.2 (beta)	CPU
Run this Command:	<pre>conda install pytorch torchvision torchaudio cpuonly -c pytorch</pre>			

- Check your CUDA version: Go to Command Prompt → type “nvidia-smi” → enter.

Basics

- Tensor is often thought of as a generalized matrix.

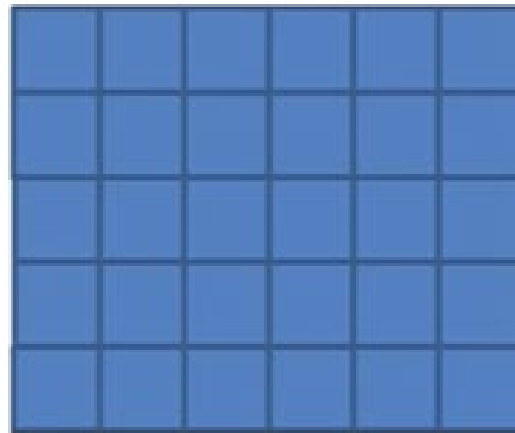


Scalar



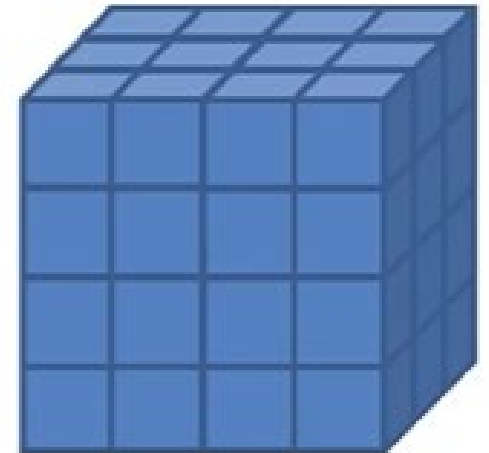
Vector

1D Tensor



Matrix

2D Tensor



Tensor

3D Tensor

Why use tensors?

- Its often easiest to arrange data sets as tensors, we've already seen data that could easily be organized into a 2D tensor.
- If we want to use image data, that will be easiest to work as 3D tensors.

Handwriting classification using ANN and CNN

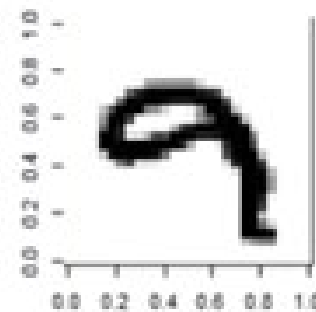
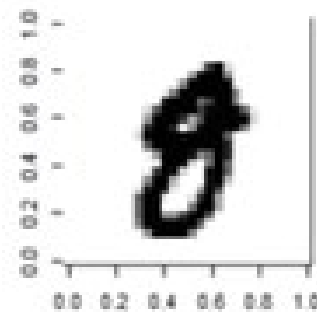
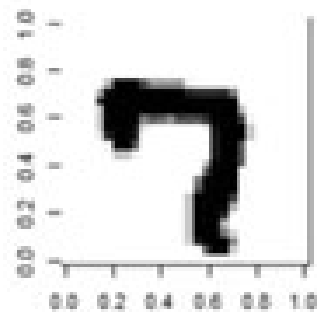
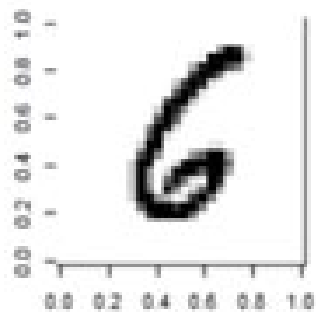
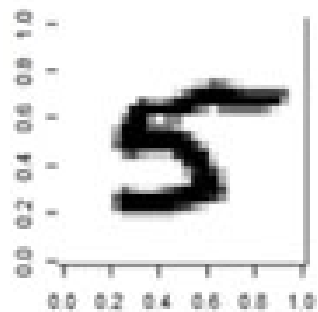
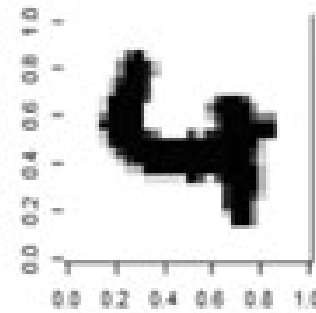
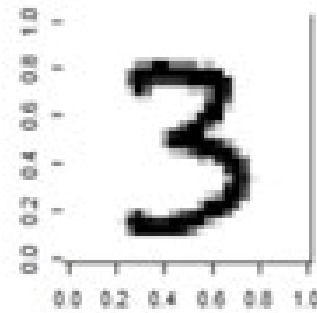
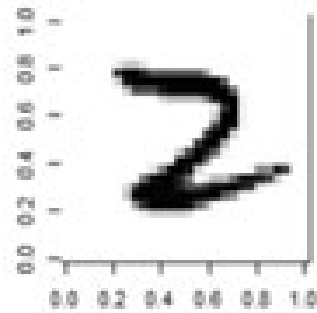
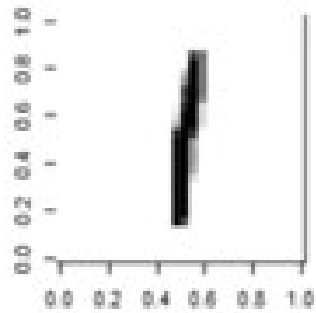
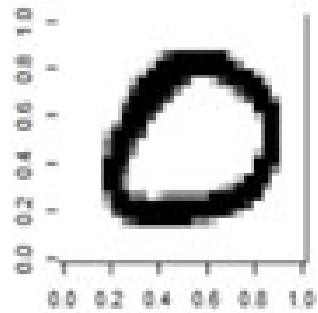
Dataset (1)

- The MNIST dataset contains handwritten single digits from 0 to 9.



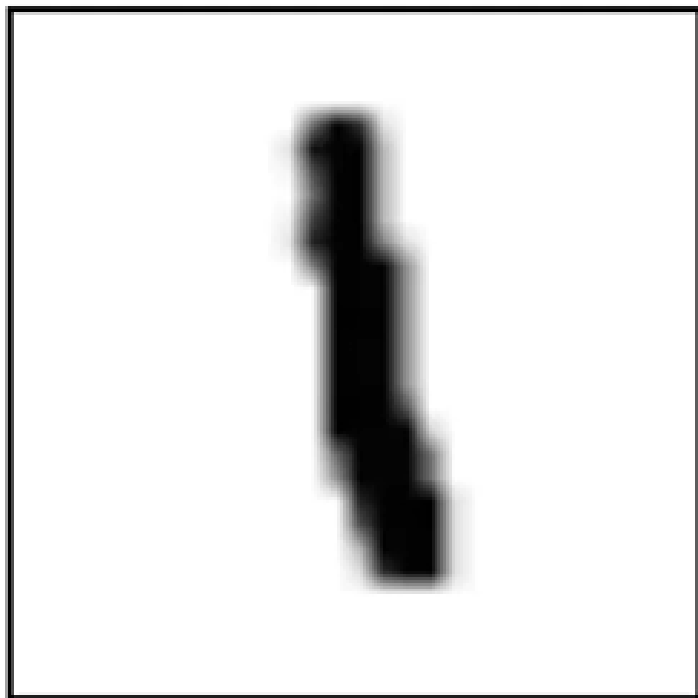
Dataset (2)

- A single digit image can be represented as an array.



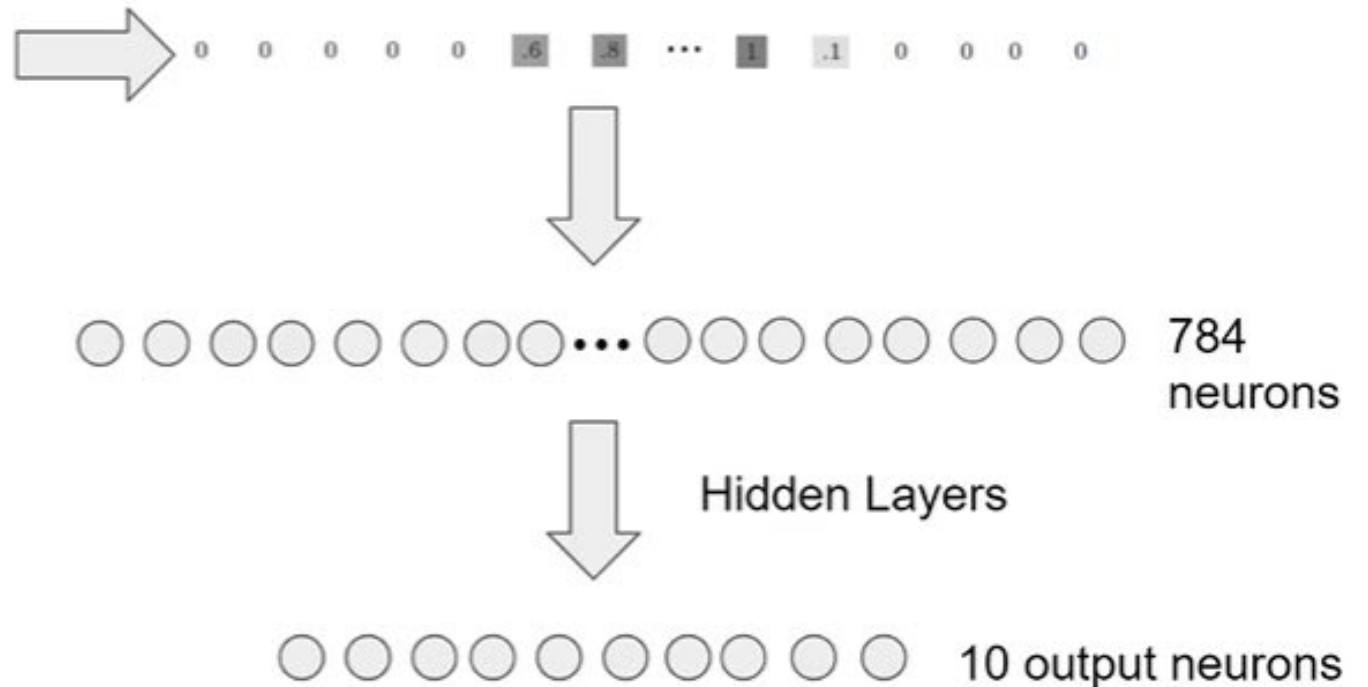
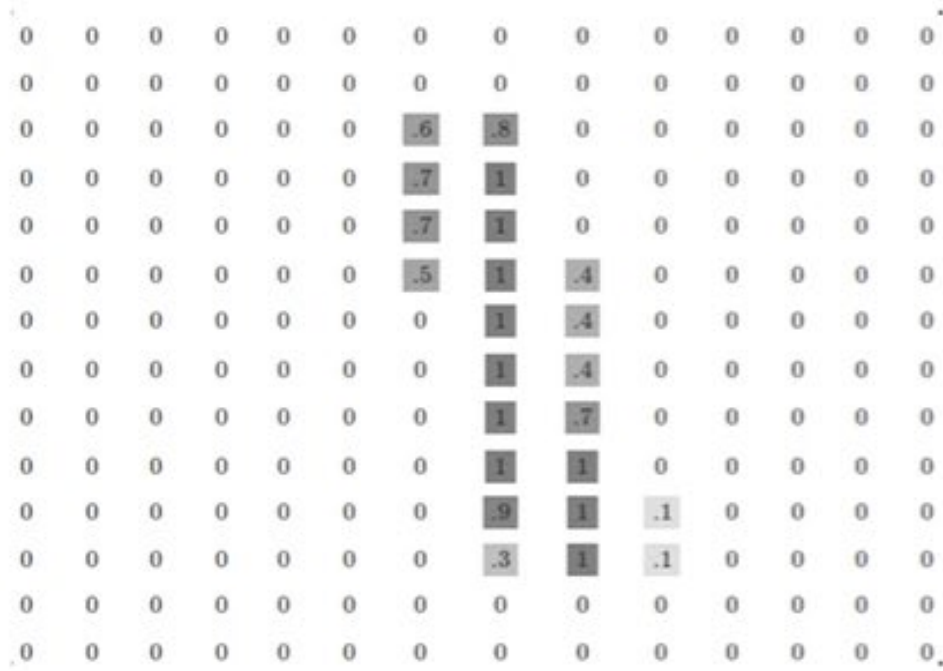
Dataset (3)

- Specifically, 28 by 28 pixels

[illegible]

ANN (1)

- In this case, to feed it into our network we will need to flatten the 28 by 28 array to 784 (28x28).



ANN (2)

- Flattening out the image ends up removing some of the 2-D information, such as the relationship of a pixel to its neighboring pixels.

MNIST USING ANN

MNIST with ANN

- Before we start working with Convolutional Neural Networks (CNN), let's model the [MNIST dataset](#) using only linear layers.
- We will use the same logic laid out in the ANN notebook → reshape the MNIST data from a 28x28 image to a flattened 1x784 vector to mimic a single row of 784 features.

1. Import Some Libraries

- Firstly, you must ensure that you have installed PyTorch successfully.
- Check version : “import torch” then “torch.__version__”

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader #Load data
from torchvision import datasets, transforms

import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
%matplotlib inline
```

2. Load the MNIST dataset

- PyTorch makes the MNIST dataset available through torchvision. The first time it's called, the dataset will be downloaded onto your computer to the path specified. From that point, torchvision will always look for a local copy before attempting another download.
- Define transform: As part of the loading process, we can apply multiple transformations (reshape, convert to tensor, normalize, etc.) to the incoming data.
- For this step, we need to convert images to tensors.

```
In [2]: transform = transforms.ToTensor()
```


2. Load the MNIST dataset: the test set

- There's a companion set of MNIST data containing 10,000 records accessible by setting `train=False`. As before, torchvision will download this.

```
In [9]: test_data = datasets.MNIST(root='../CNN/Data', train=False, download=True, transform=transform)
test_data
```

```
Out[9]: Dataset MNIST
        Number of datapoints: 10000
        Root location: ../CNN/Data
        Split: Test
        StandardTransform
        Transform: ToTensor()
```

2. Load the MNIST dataset: preview a training data (2)

- Another way to see the label :

```
In [11]: image, label = train_data[0]
print('Shape:', image.shape, '\nLabel:', label)

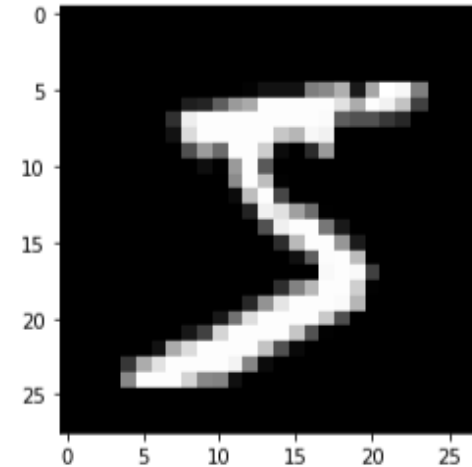
Shape: torch.Size([1, 28, 28])
Label: 5
```

- “1” in the torch.size → meaning that the image only has 1 layer → grayscale.
- We can see the preview label = 5.

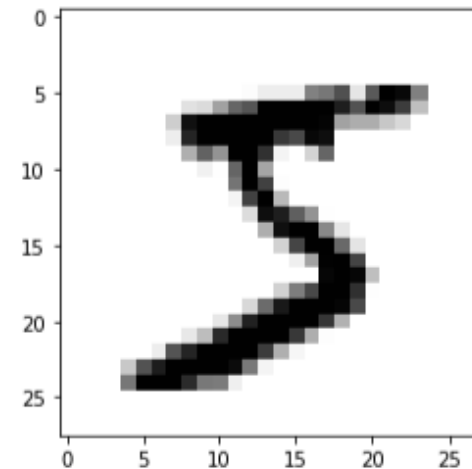
2. Load the MNIST dataset: view the image

- Matplotlib can interpret pixel values through a variety of colormaps.
- Reshape the image by 28x28 to make 2D matrix.

```
In [12]: plt.imshow(train_data[0][0].reshape((28,28)), cmap="gray");
```



```
In [13]: plt.imshow(train_data[0][0].reshape((28,28)), cmap="gist_yarg");
```



3. Batch loading with DataLoader (1)

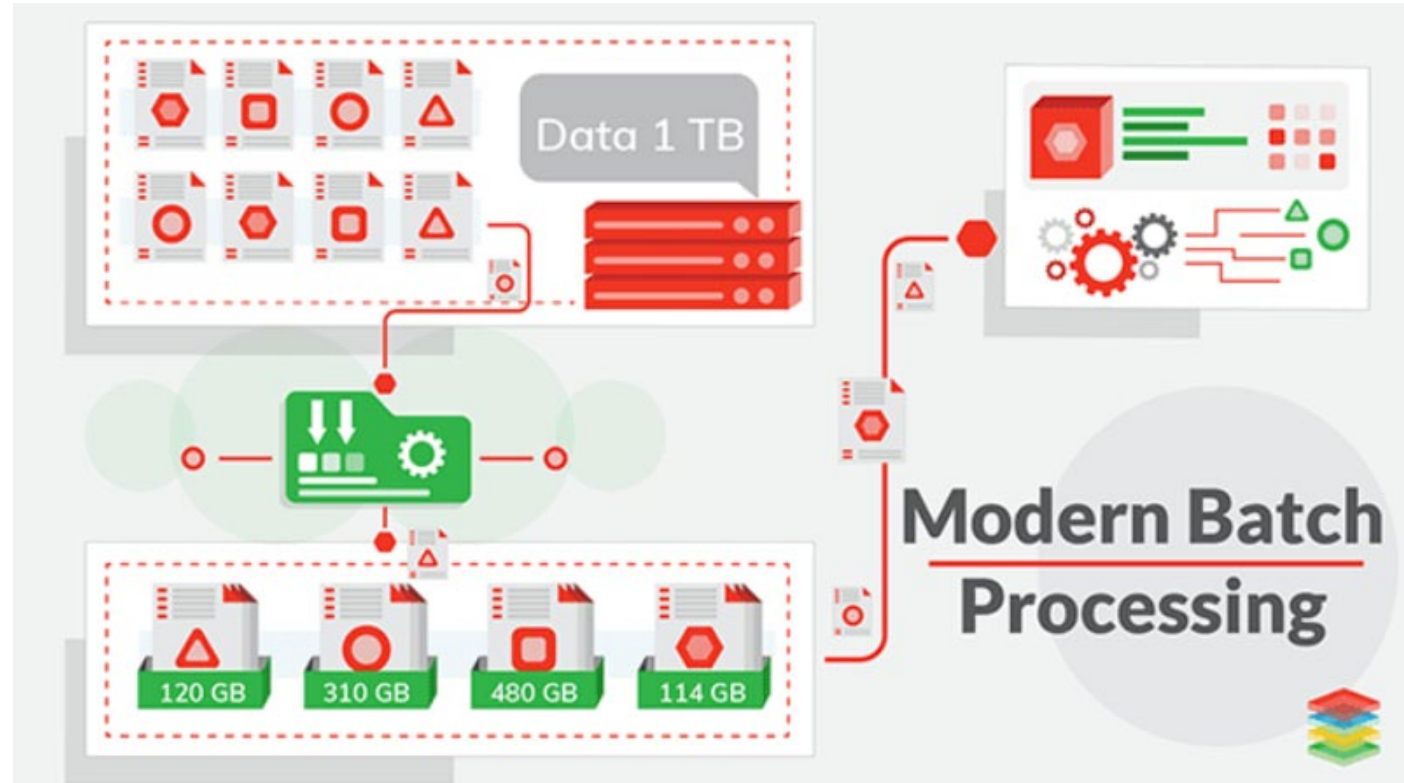
- The training set contains 60,000 records. If we look ahead to our model we have 784 incoming features.
- We try to set hidden layers of 120 neurons and another hidden layer of 84 neurons (2 hidden layers), and 10 output features. Including the bias terms for each layer, the total number of parameters being trained is:

$$\begin{aligned} &\text{Weights} \quad \text{bias} \\ &(784 \times 120) + 120 + (120 \times 84) + 84 + (84 \times 10) + 10 = \\ &\quad 94080 + 120 + 10080 + 84 + 840 + 10 = 105,214 \end{aligned}$$

- For this reason, it makes sense to load training data in batches using DataLoader
(<https://pytorch.org/docs/stable/data.html#torch.utils.data.DataLoader>)

3. Batch loading with DataLoader (2)

- Batch Processing is a technique for processing large amounts of data in a repeatable manner.
- Simply described, batch processing entails users collecting and storing data in batches during the course of a “batch window”.
- This saves time and enhances data processing efficiency, allowing researchers to handle enormous volumes of data and analyze it rapidly.



3. Batch loading with DataLoader (3)

```
In [15]: torch.manual_seed(101) # for consistent results

train_loader = DataLoader(train_data, batch_size=100, shuffle=True)

test_loader = DataLoader(test_data, batch_size=500, shuffle=False)
```

- `Torch.manual_seed()` → we want to make sure that the random shuffle is fixed when we run the program, not change → changing SEED can change the results or the output of a neural network. Check the number of SEED using “`torch.manual_seed(101)`” then type “`torch.rand(2,2)`” for displaying into 2x2 matrix.
- The `batch_size` is the number of records to be processed at a time. If it's not evenly divisible into the dataset, then the final batch contains the remainder. Ex: `batch_size=100` → feeding 100 images as 1 batch at a time into a network.
- Setting `shuffle` to `True` means that the dataset will be shuffled after each epoch.
- `Train_loader` → `shuffle = true` because we want to get minimum error and we don't know the best sequence of images that can reach this goal.
- `Test_loader` → `shuffle = false` because we will not adjust the weights and bias, so it doesn't matter as far as the shuffling.

3. Batch loading with DataLoader: View a batch of images

- Create a grid of images using `torchvision.utils.make_grid` → because we use `DataLoader`, it shows a grid of images.
- Since there is 60,000 images and the batch size = 100, then the number of looping = $60,000 / 100 = 60$ times → so if we want to show just 1 loop as first batch, add “break” after looping.
- Because of : 
- meaning that `torch.size(100, color, width, height)`, then we need to transpose the images from CWH to WCH.

`images.shape`

`torch.Size([100, 1, 28, 28])`

```
In [26]: from torchvision.utils import make_grid
np.set_printoptions(formatter=dict(int=lambda x: f'{x:4}')) # format to widen the printed array

# Grab the first batch of images
for images, labels in train_loader:
    break

# Print the first 12 Labels from 100 Labels (remember we set 1 batch = 100 images)
print('Labels: ', labels[:12].numpy())

# Print the first 12 images
im = make_grid(images[:12], nrow=12)
plt.figure(figsize=(10,4))
# We need to transpose the images from CWH (Color, Width, Height) to WHC (Width, Height, Color)
plt.imshow(np.transpose(im.numpy(), (1, 2, 0)));
```

Labels: [2 0 9 5 8 9 9 8 6 4 7 2]



3. Batch loading with DataLoader: Define the model (1)

- We will use fully connected layers to develop a multilayer perceptron.
- The input size is 784 → we flatten the incoming 28x28 tensors.
- The output size represents the 10 possible digits.
- Set the hidden layers to [120, 84]. After this experiment, we can try another number of neurons.

```
In [27]: class MultilayerPerceptron(nn.Module):
          def __init__(self, in_sz=784, out_sz=10, layers=[120,84]):
              super().__init__()
              self.fc1 = nn.Linear(in_sz, layers[0]) #input layer, layers[0]==>
              self.fc2 = nn.Linear(layers[0], layers[1]) #hidden layer 1
              self.fc3 = nn.Linear(layers[1], out_sz) #hidden layer 2 to output

          def forward(self, X):
              X = F.relu(self.fc1(X))
              X = F.relu(self.fc2(X))
              X = self.fc3(X)
              return F.log_softmax(X, dim=1)
```

3. Batch loading with DataLoader: Define the model (2)

- Torch.nn.Linear → to calculate linear combiner and to apply a linear transformation to the incoming data: $y = xA^T + b$
- Remember when you attended DSC class in the 6th semester!

- ▶ **Step 1** : Set $k = 1$, initialize the synaptic weight vector $w(k=1)$ and select values for μ_0 and τ .

- ▶ **Step 2** : Compute learning rate parameter $\mu(k)$:

$$\mu(k) = \frac{\mu_0}{1 + \frac{k}{\tau}}$$

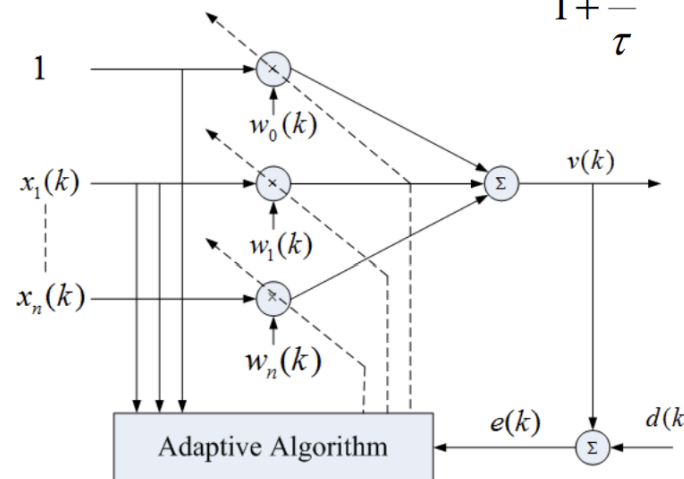
- ▶ **Step 3** : Compute $v(k)$:

$$v(k) = \sum_{i=0}^n w_i(k)x_i(k)$$

- ▶ **Step 4** : Compute error :

$$e(k) = d(k) - v(k)$$

- ▶ **Step 5** : Update the synaptic weights : $w_i(k+1) = w_i(k) + \mu(k)e(k)x_i(k)$
for $i = 0, 1, 2, \dots, n$.
- ▶ **Step 6** : if convergence is achieved then stop, else set $k \leftarrow k+1$ then go to step 2.



3. Batch loading with DataLoader: Define the model (3)

- In this model, we set activation function to Rectified Linear Unit (ReLU) => because in this case we have multiclass classification problem.
- ReLU for short is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero.
- ReLU has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance.

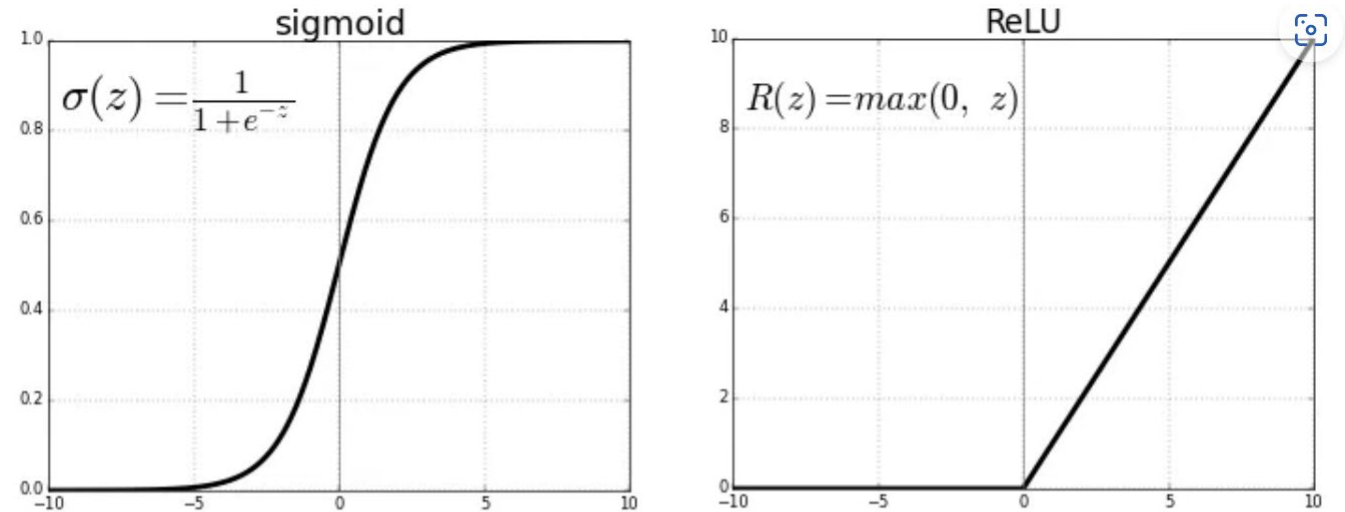


Fig: ReLU v/s Logistic Sigmoid

As you can see, the ReLU is half rectified (from bottom). $f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero.

Range: [0 to infinity)

3. Batch loading with DataLoader: Define the model (4)

- After the output layer, we need to solve the problem of multiclass classification. The problem is the raw outputs of the NN are very difficult to interpret, so we need activation function to simplify the interpretation of output layer.
- The **softmax activation function** transforms the raw outputs of the neural network into a vector of probabilities, essentially a probability distribution over the input classes.
- Use “F.log_softmax(X, dim(int))” → **dim(int)** is a dimension along which log_softmax will be computed. In this case, we set one dimension.

```
In [27]: class MultilayerPerceptron(nn.Module):
def __init__(self, in_sz=784, out_sz=10, layers=[120,84]):
    super().__init__()
    self.fc1 = nn.Linear(in_sz, layers[0]) #input layer, layers[0]==>
    self.fc2 = nn.Linear(layers[0], layers[1]) #hidden layer 1
    self.fc3 = nn.Linear(layers[1], out_sz) #hidden layer 2 to output

def forward(self, X):
    X = F.relu(self.fc1(X)) #forward input layer to hidden
    X = F.relu(self.fc2(X)) #result from previous layer, p
    X = self.fc3(X) #forward process from hidden layer 2 to t
    return F.log_softmax(X, dim=1) #dim=dimension
```

```
torch.manual_seed(101)
model = MultilayerPerceptron()
model
```

```
MultilayerPerceptron(
  (fc1): Linear(in_features=784, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```


Softmax Activation Function (1)

- Ex: Given a dataset containing images of pandas, seals, and ducks. We'd like to train a neural network to predict whether a previously unseen image is that of a seal, a panda, or a duck.
- Notice how the input class labels below are one-hot encoded, and the classes are *mutually exclusive*. In this context, mutual exclusivity means that a given image can only be *one* of {seal, panda, duck} at a time.

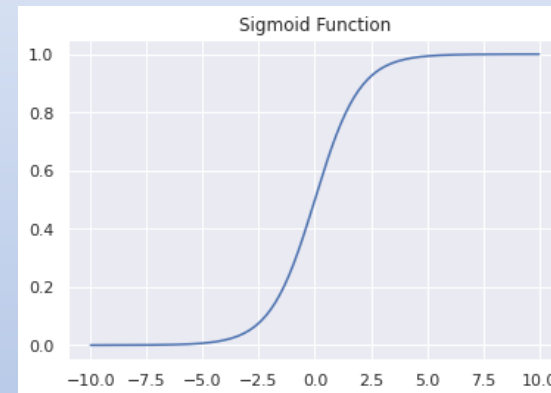
Class	Value	One-Hot Encoding
0		$[1, 0, 0]$
1		$[0, 1, 0]$
2		$[0, 0, 1]$

Limitations of the Sigmoid Function

- Mathematically, the sigmoid activation function is given by the following equation, and it squishes all inputs onto the range $[0, 1]$.

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

where, $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$
and $\sigma(z) \rightarrow 1$ as $z \rightarrow \infty$



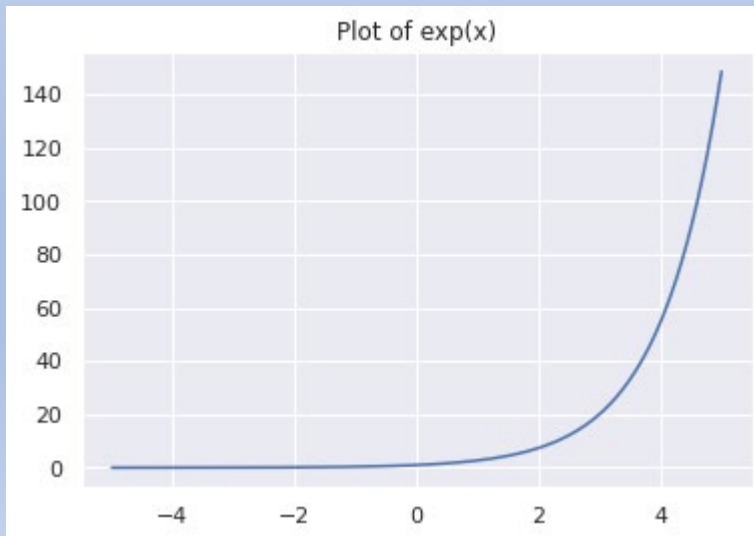
- Let's go back to the example of classifying whether an input image is that of a panda or not.
- In this case, let z be the raw output of the neural network. If $\sigma(z)$ is the probability that the given image belongs to class 1 (is a panda) $\rightarrow$ then $1 - \sigma(z)$ is the probability that the given image does not belong to class 1 and is not a panda.
- Therefore, we can suppose that $\sigma(z)$ as a probability score.
- However, this won't quite work when we have more than two classes.

Softmax Activation Function (2)

- The softmax activation function takes in a vector of raw outputs of the neural network and returns a vector of probability scores.
- The equation of the softmax function is given as follows:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

- $\mathbf{z}$ is the vector of raw outputs from the neural network
- The value of $e \approx 2.718$
- The i -th entry in the softmax output vector $\text{softmax}(\mathbf{z})$ can be thought of as the predicted probability of the test input belonging to class i .



From the plot of e^x , we can see that, regardless of whether the input x is positive, negative, or zero, e^x is always a positive number.

Softmax Activation Function (3)

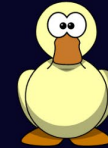
- Recall that in the example, $N = 3$ as we have 3 classes: {seal, panda, duck}, and the valid indices are 0, 1, and 2. Suppose we're given the vector $\mathbf{z} = [0.25, 1.23, -0.8]$ of raw outputs from the neural network.
- Let's apply the softmax formula on the vector $\mathbf{z}$:
- The entries of the softmax output sum up to 1: $0.664 + 0.249 + 0.087 = 1$.

$$\begin{aligned} Pr[y_i = seal] &= softmax(\mathbf{z})_0 \\ &= \frac{e^{0.25}}{e^{0.25} + e^{1.23} + e^{-0.8}} = 0.249 \end{aligned}$$



$$\begin{aligned} Pr[y_i = panda] &= softmax(\mathbf{z})_1 \\ &= \frac{e^{1.23}}{e^{0.25} + e^{1.23} + e^{-0.8}} = 0.664 \end{aligned}$$

$$\begin{aligned} Pr[y_i = duck] &= softmax(\mathbf{z})_2 \\ &= \frac{e^{-0.8}}{e^{0.25} + e^{1.23} + e^{-0.8}} = 0.087 \end{aligned}$$



Softmax Activation Function (4)

- In the vector $\text{softmax}(z) = [0.664, 0.294, 0.087]$, 0.664 at index 1 is the largest value. This means there's a 66.4% chance that the given image belongs to class 1, which is a class panda.
- And the input image has a 29.4% chance of being a seal and around 8.7% chance of being a duck.
- Therefore, applying softmax gives instant interpretability, as we know how likely the test image is to belong to each of the 3 classes. In this particular example, it's highly likely to be a panda and least likely to be a duck.

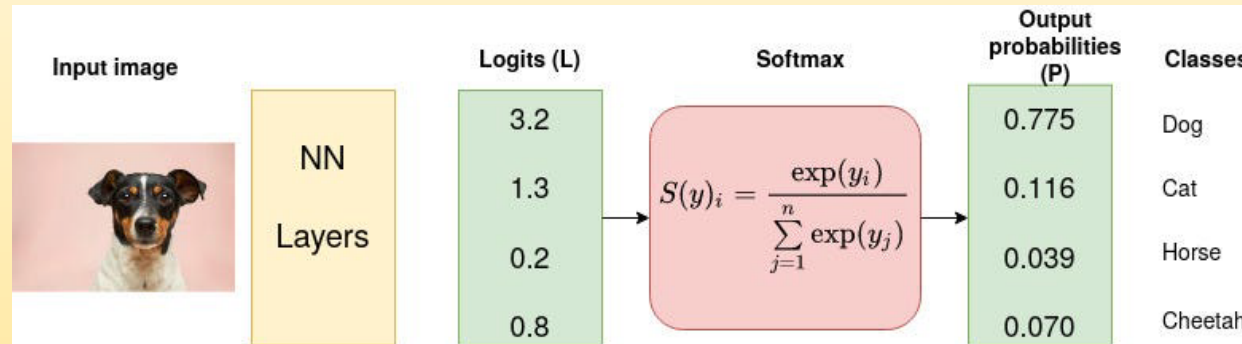
3. Batch loading with DataLoader: Define loss function & optimizer

```
In [29]: criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

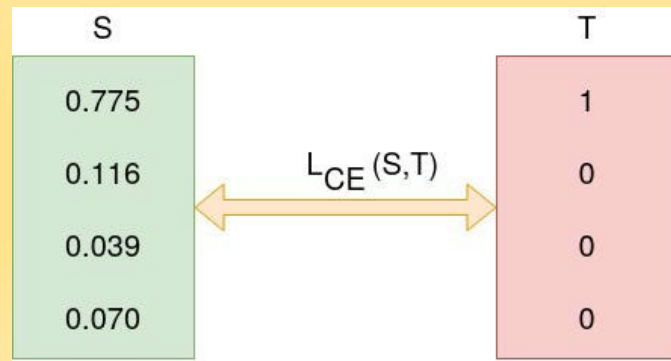
- When working on a Machine Learning or a Deep Learning Problem, loss/cost functions are used to optimize the model during training.
- The objective is almost always to minimize the loss function.
- The lower the loss the better the model. Cross-Entropy loss is a most important cost function. It is used to optimize classification models.

Cross-Entropy Loss Function

- Consider a 4-class classification task where an image is classified as either a dog, cat, horse or cheetah.



- In the above Figure, Softmax converts logits into probabilities. The purpose of the Cross-Entropy is to take the output probabilities (P) and measure the distance from the truth values (as shown in Figure below).



For the example above the desired output is [1,0,0,0] for the class dog but the model outputs [0.775, 0.116, 0.039, 0.070] .

$$\begin{aligned} L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\ &= - [1 \log_2(0.775) + 0 \log_2(0.116) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\ &= - \log_2(0.775) \\ &= 0.3677 \end{aligned}$$

Optimizer

- [GitHub - Jaewan-Yun/optimizer-visualization: Visualize Tensorflow's optimizers.](#)
- [Gentle Introduction to the Adam Optimization Algorithm for Deep Learning - MachineLearningMastery.com](#)
- [Intuition of Adam Optimizer - GeeksforGeeks](#)

3. Batch loading with DataLoader: Flatten the training data

- The batch tensors fed in by DataLoader have a shape of [100, 1, 28, 28].
- We have to convert the shape to one dimension (784) using “.view()”

```
In [30]: # Load the first batch, print its shape
for images, labels in train_loader:
    print('Batch shape:', images.size())
    break
```

```
Batch shape: torch.Size([100, 1, 28, 28])
```

```
In [31]: images.view(100, -1).size()
```

```
#-1 , meaning grab all after array 100 until sequence before -1, and combine them into a single dimension
```

```
Out[31]: torch.Size([100, 784])
```

3. Batch loading with DataLoader: Train the model (1)

- Set total time for training process :
- Define some parameters and make looping for each batch.

```
import time
start_time = time.time()

total_time = time.time() - start_time
print(f'Duration: {total_time/60} mins')
```

```
#VARIABLES
epochs = 5
train_losses = []
test_losses = []
train_correct = []
test_correct = []

for i in range(epochs):
    trn_corr = 0 #training correct
    tst_corr = 0 #test correct

    # TRAIN batches, b = batches
    for b, (X_train, y_train) in enumerate(train_loader):
        b+=1

        # Apply the model
        y_pred = model(X_train) # we don't flatten X-train here, keep data in 2D image
        loss = criterion(y_pred, y_train) #cek selisih error
```

3. Batch loading with DataLoader: Train the model (2)

- In this case, the output model consists of 10 neurons with each probability [0.1, 0.0, ..., 0.8], predicting number 0 to 9.
- Calculate the number of correct prediction:
`predicted = torch.max(y_pred.data, 1)[1]`
- The code above transforms the predicted probability into one hot encoding, `y_pred.data, 1` → 1 meaning that actual predicted value.
- `torch.max()` function returns a tensor of maximum values, and a tensor of the indices where the max values were found. In the code, we're asking for the index positions of the maximum values along dimension 1. In this way, we can match predictions up to image labels.

3. Batch loading with DataLoader: Train the model (3)

- Calculating how many matches from predicted values is equal to training value of a batch:

```
batch_corr = (predicted == y_train).sum()
```

- The number of correct while training process:

```
trn_corr += batch_corr
```

- Updating parameters:

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

3. Batch loading with DataLoader: Train the model (4)

- Calculating the accuracy per epoch → $\text{accuracy} = \text{training correct} * 100\% / \text{batch size} * b$ → in this case, we set batch size = 100.

```
# Print results per epoch:
if b%200 == 0:
    print(f'epoch: {i:2}  batch: {b:4} [{100*b:6}/60000]  loss: {loss.item():10.8f}
accuracy: {trn_corr.item()*100/(100*b):7.3f}%')
```

- Updating train loss and accuracy for the epoch:

```
# Update train loss & accuracy for the epoch
train_losses.append(loss.item())
train_correct.append(trn_corr.item())
```

3. Batch loading with DataLoader: Run the testing batches

```
# Run the testing batches
with torch.no_grad(): #don't update weight and bias in test data
    for b, (X_test, y_test) in enumerate(test_loader):

        # Apply the model
        y_val = model(X_test.view(500, -1)) # Here we flatten X_test,

        # Calculating the number of correct predictions
        predicted = torch.max(y_val.data, 1)[1]
        tst_corr += (predicted == y_test).sum()

# Update test loss & accuracy for the epoch
loss = criterion(y_val, y_test)
test_losses.append(loss)
test_correct.append(tst_corr)
```

4. Evaluation: Comparisons of loss and accuracy (1)

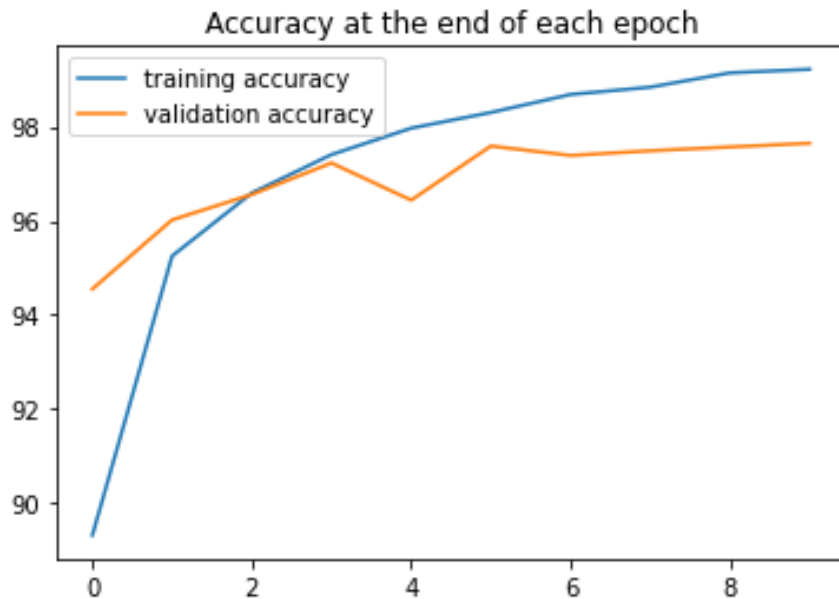
- In a real world situation, test/validation loss should be higher than the training loss.
- Plot epochs vs loss (error):
- Based on the results, test data would start overfitting at epoch more than 8 to training data.

```
plt.plot(train_losses, label='training loss')  
plt.plot(test_losses, label='validation loss')  
plt.title('Loss at the end of each epoch')  
plt.legend();
```



4. Evaluation: Comparisons of loss and accuracy (2)

```
plt.plot([t/600 for t in train_correct], label='training accuracy') #600 because training data consist of 60,000/100 (ba  
plt.plot([t/100 for t in test_correct], label='validation accuracy') #100 because test data 10,000/100 = 100  
plt.title('Accuracy at the end of each epoch')  
plt.legend();
```



4. Evaluation: Evaluate Test Data

```
In [24]: # Extract the data all at once, not in batches
test_load_all = DataLoader(test_data, batch_size=10000, shuffle=False)
```

```
In [25]: with torch.no_grad():
    correct = 0
    for X_test, y_test in test_load_all:
        y_val = model(X_test.view(len(X_test), -1)) # pass in a flattened view of X_test
        predicted = torch.max(y_val, 1)[1]
        correct += (predicted == y_test).sum()
    print(f'Test accuracy: {correct.item()}/{len(test_data)} = {correct.item()*100/(len(test_data)):7.3f}%')
```

Test accuracy: 9765/10000 = 97.650%

4. Evaluation: Confusion Matrix

```
# print a row of values for reference
np.set_printoptions(formatter=dict(int=lambda x: f'{x:4}')) #x:4 => giving space each data
print(np.arange(10).reshape(1,10))
print()

# print the confusion matrix
print(confusion_matrix(predicted.view(-1), y_test.view(-1))) #The view(-1) operation flattens the tensor,
```

```
[[ 0  1  2  3  4  5  6  7  8  9]]

[[ 971  0  1  1  0  1  1  1  3  5]
 [  0 1122  2  0  1  1  3  8  0  3]
 [  2  3 1017  6  6  0  2 12  2  0]
 [  1  3  3 993  0 19  1  5  3  9]
 [  0  0  1  0 950  1  2  0  0  7]
 [  0  0  0  1  0 850  1  1  2  8]
 [  3  1  2  1  9  9 947  0  3  1]
 [  0  0  3  3  2  0  0 990  2  0]
 [  3  6  2  3  2 10  1  6 956  7]
 [  0  0  1  2 12  1  0  5  3 969]]
```

This shows that the model had the greatest success with ones, twos and sevens, and the lowest with fives, sixes and eights.

Assignment

- Try to design another model of ANN to classify MNIST.
- Realize your program using GUI (Streamlit).



MNIST USING CNN

Weakness of ANN

- Large amount of parameters (over 100,000 for tiny 28 by 28 images)
- Input features = 700, hidden 1 = 20, hidden 2 = 10, output = 5
- Parameters = $(700 \times 20) + 20 + (20 \times 10) + 10 + (10 \times 5) + 5 = \dots$
- We will lose all 2D information by flattening out the image.
- Will work only on very similar, well centered images → for example if we were to take a 28x28 image of a handwritten digit, but the digit is not centered, maybe the digit is started on the right and feed into ANN, we will not get good results → so, if we use ANN, we have to make sure that the digit are perfectly centered.

CNN

- CNN helps reduce parameters by focusing on local connectivity.
- Not all neurons will be fully connected → neurons are only connected to a subset of local neurons in the next layer, and becoming the filters.

ANN → Fully connected, lots of parameters!

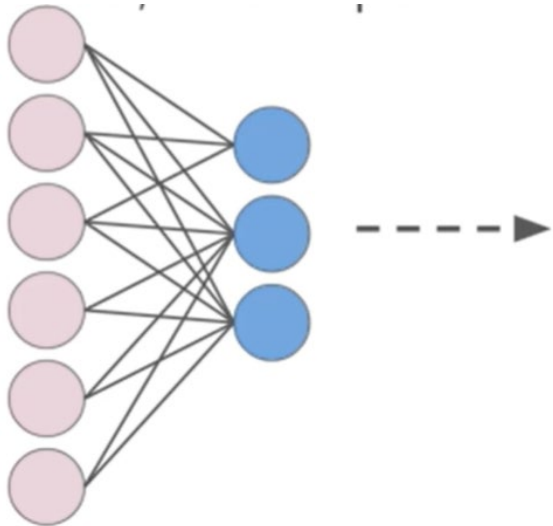


Illustration of CNN in 1D → Localized connections

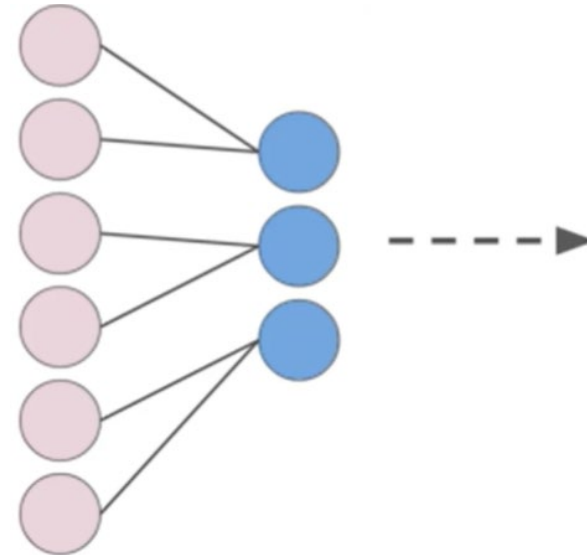


Illustration of 2D CNN (1)

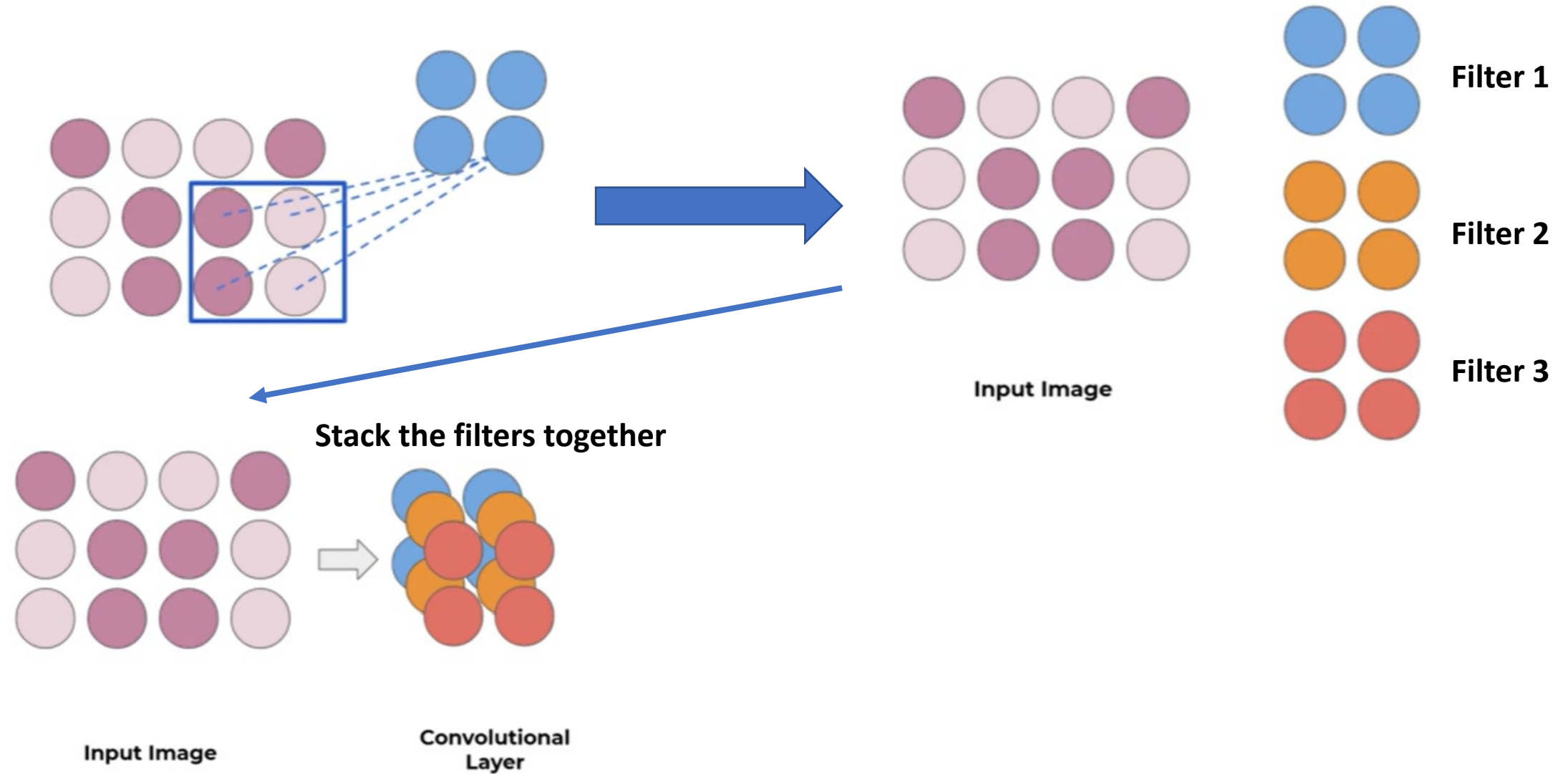
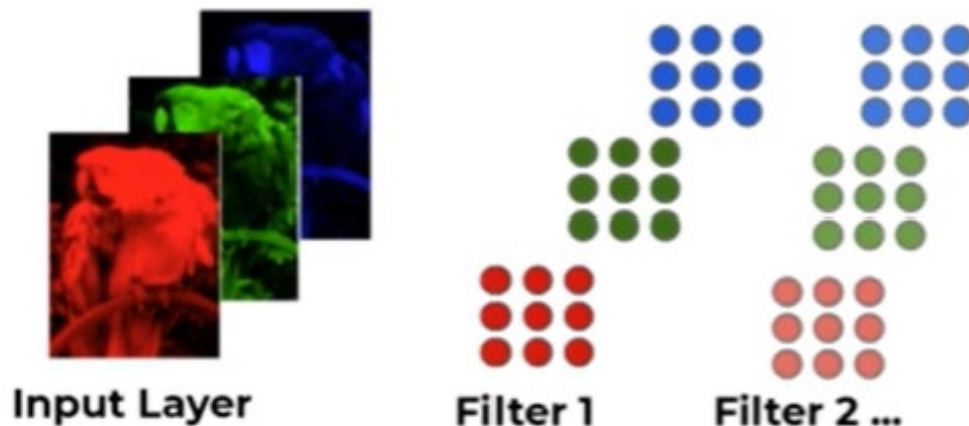


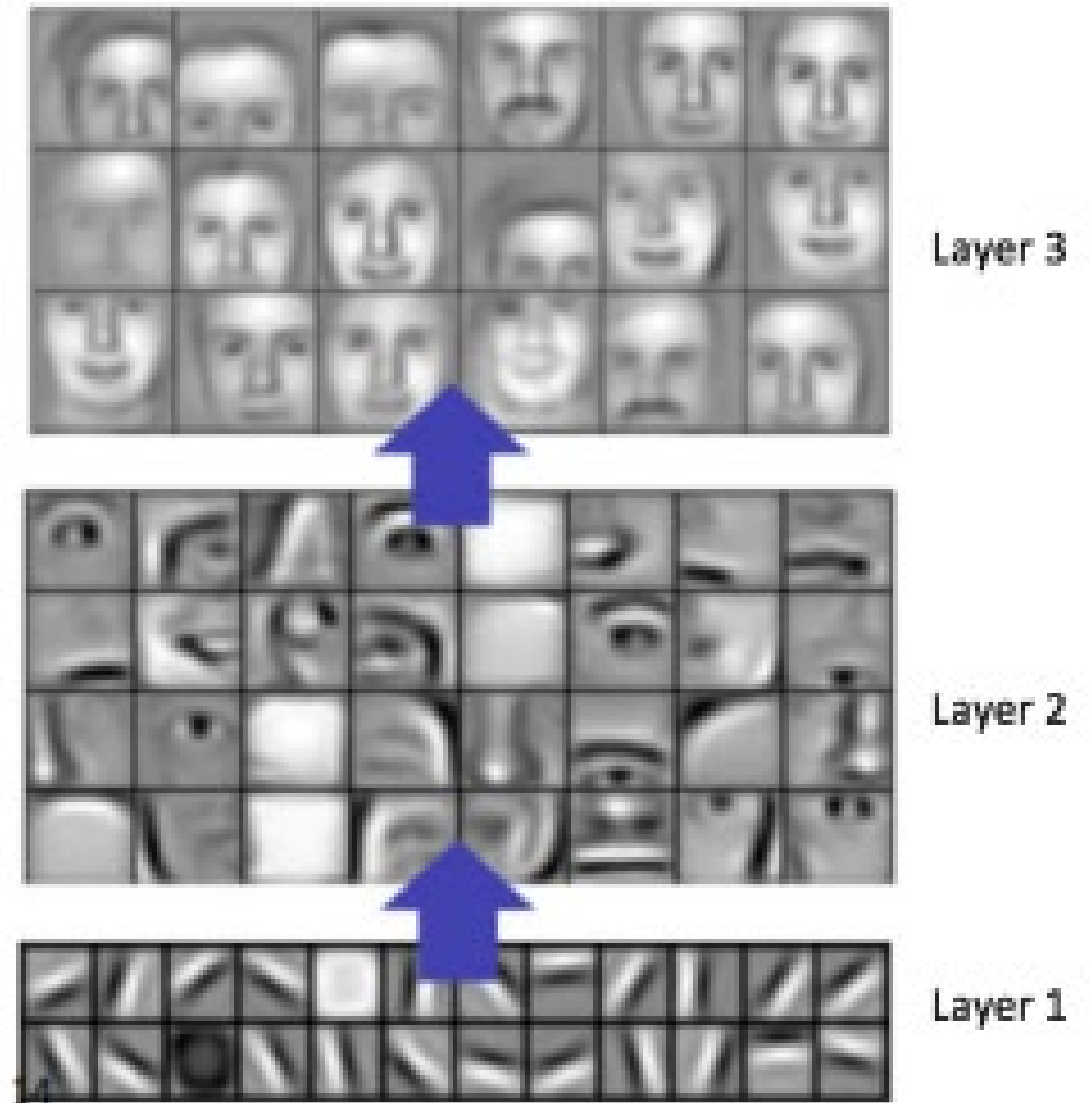
Illustration of 2D CNN (2)

- But the previous example was just in 2D for grayscale images.
- What about color images?
- Color images can be thought of as 3D Tensors consisting of Red, Green, and Blue color channels.
- We end up 3D filter values for each color channel.



The power combination in Neural Network ==> Feature Extraction

- Goal awal → bagaimana caranya kombinasi citra menghasilkan fitur deteksi?

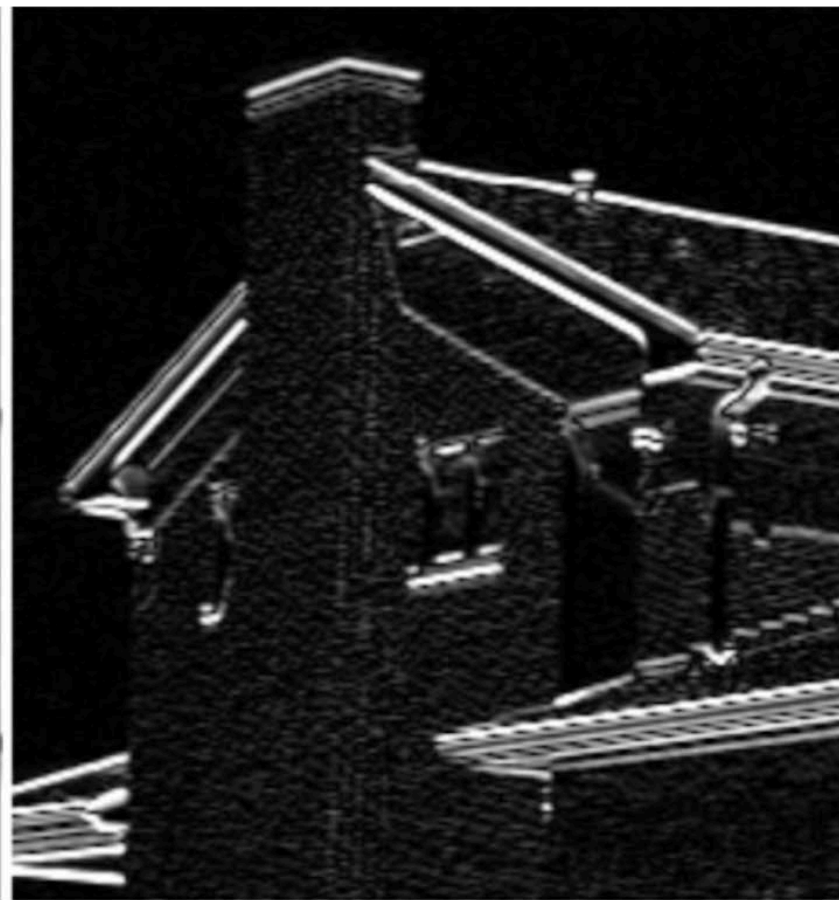
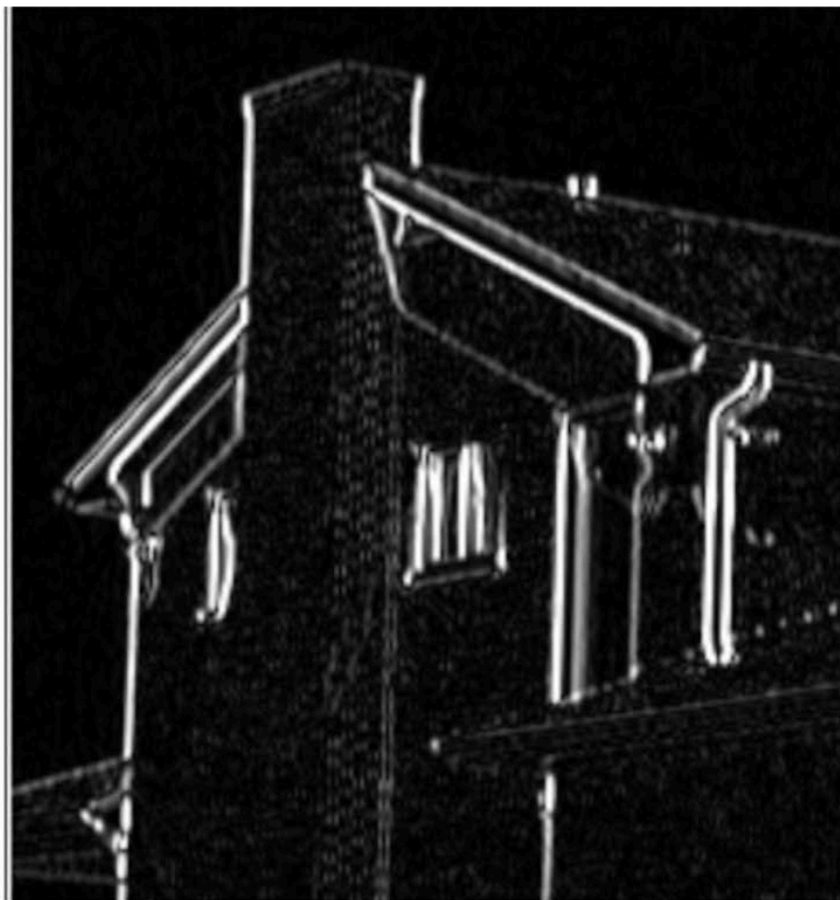


Our main goal

Original

Vertical Edge

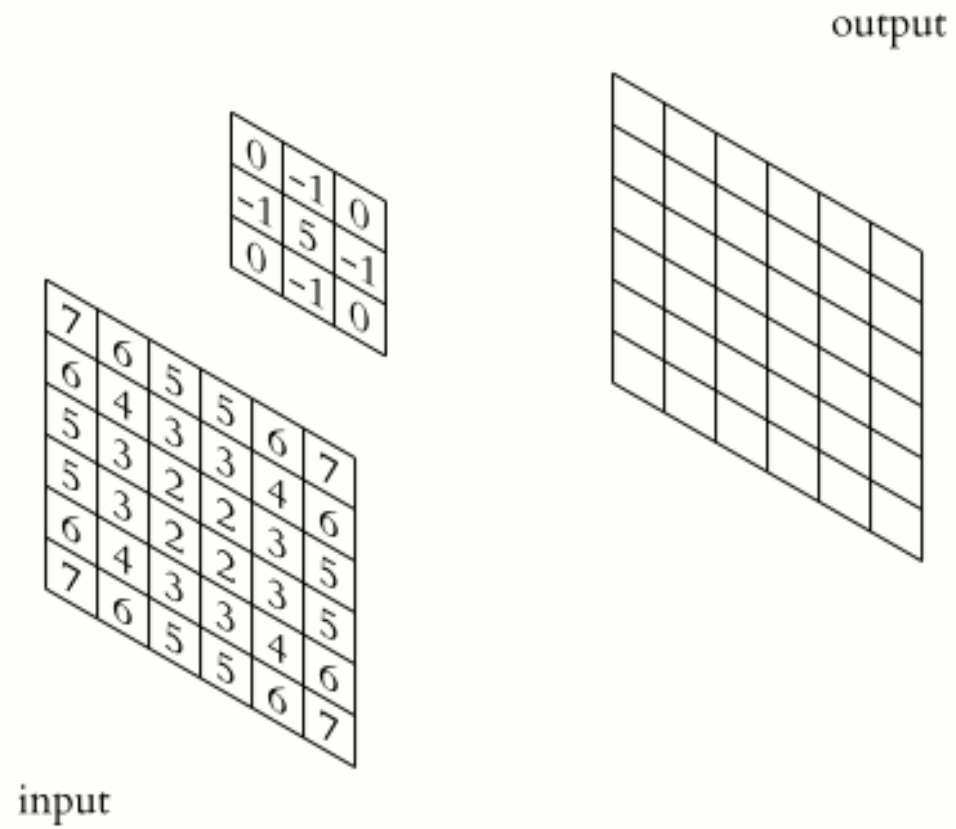
Horizontal Edge



Concept 1: Filter and Edge Detection

- <http://matlabtricks.com/post-5/3x3-convolution-kernels-with-online-demo>
- Smoothing
- Sharpening
- Konvolusi citra dengan kernel

Convolution

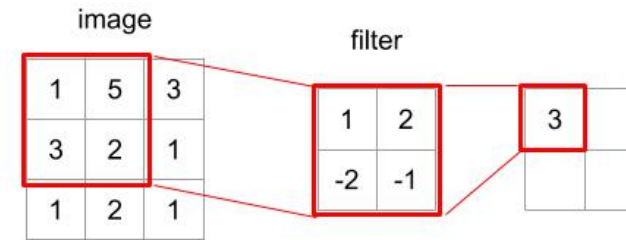


Concept 2: Convolution

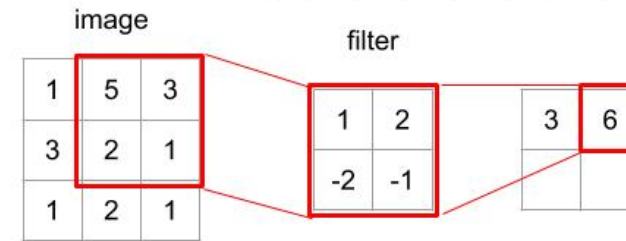
- Menerapkan konvolusi di citra → kombinasi linear yg menghasilkan fitur baru.
- Hasil konvolusi → feature map → **resolusi lebih kecil.**

image			filter	
1	5	3	1	2
3	2	1	2	1
7	5	5		

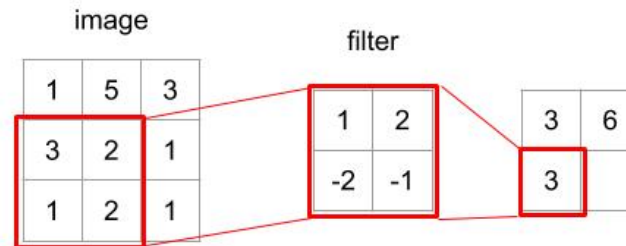
$$(1*1) + (5*2) + (3*-2) + (2*-1) = 3$$



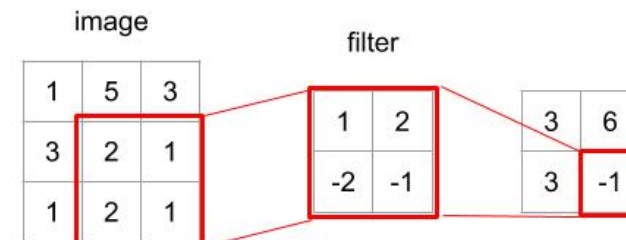
$$(5*1) + (3*2) + (2*-2) + (1*-1) = 6$$



$$(3*1) + (2*2) + (1*-2) + (2*-1) = 3$$



$$(2*1) + (1*2) + (2*-2) + (1*-1) = -1$$



Concept 3: Stride (Langkah)

- Stride → berapa piksel kernel akan digeser dari citra originalnya.
- **Stride yang besar menyebabkan hasil konvolusi yang lebih kecil.**

stride = 1

1	4	3	5	3
2	4	5	3	2
4	5	6	1	5
4	3	3	3	2
4	3	5	4	5



stride = 2

1	4	3	5	3
2	4	5	3	2
4	5	6	1	5
4	3	3	3	2
4	3	5	4	5



Concept 4: Padding

- Padding biasa digunakan untuk menjamin ukuran sama seperti ukuran semula.
- Contoh zero padding :

padding = 1

0	0	0	0	0
0	4	5	3	0
0	5	6	1	0
0	3	3	3	0
0	0	0	0	0

padding = 2

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	4	5	3	0	0
0	0	5	6	1	0	0
0	0	3	3	3	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Padding menjaga ukuran citra sama

- Contoh : citra 3x3, padding 1 layer, setelah konvolusi tetap 3x3.

stride = 1
padding = 1
kernel = 3x3

0	0	0	0	0
0	4	5	3	0
0	5	6	1	0
0	3	3	3	0
0	0	0	0	0

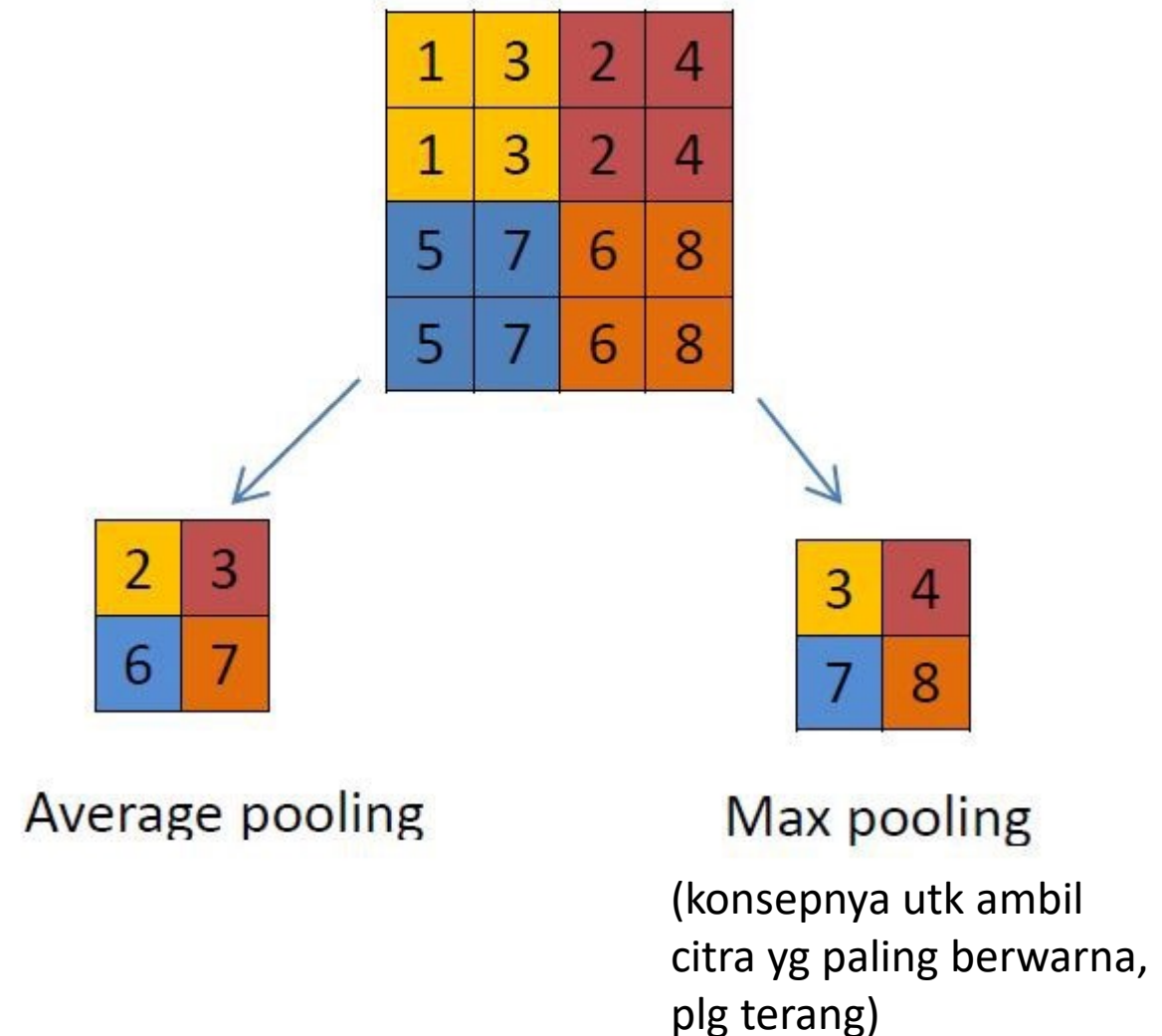


Convolution Animations

- [GitHub - vdumoulin/conv_arithmetic: A technical report on convolution arithmetic in the context of deep learning](#)

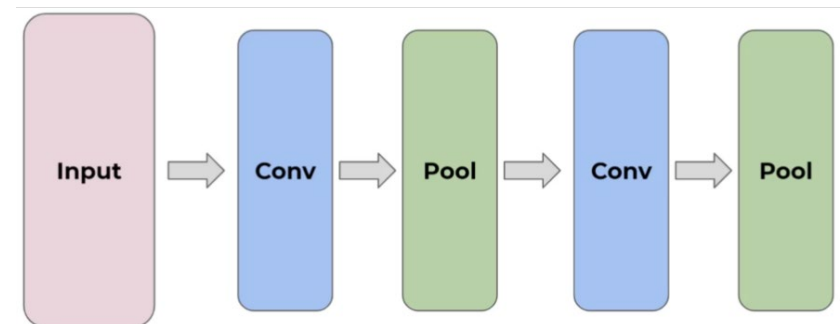
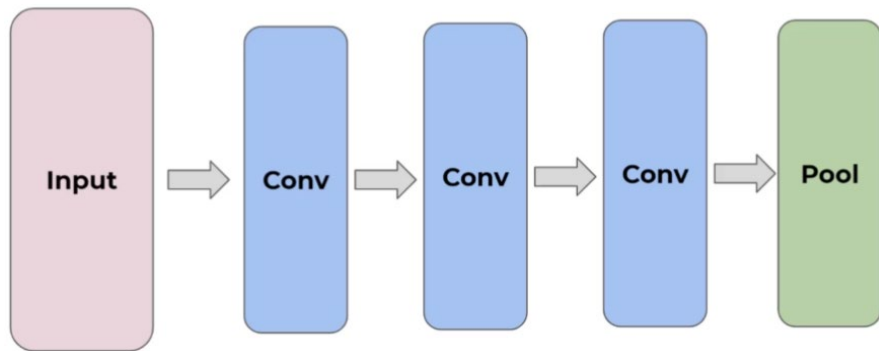
Concept 5: Pooling

- Setelah dilakukan proses konvolusi, biasanya ukuran citra masih terlalu besar, padahal akhir dari CNN adalah **proses flattening**.
- Katakanlah ukuran citra setelah konvolusi 100x100 piksel → artinya ada 10.000 node untuk flattening → komputasinya berat → solusinya dilakukan Pooling.
- Konsepnya → ambil intensitas citra yang berwarna → karena mengandung banyak informasi citra.



Pooling Layers: Purpose

- Even with local connectivity, when dealing with color images and possibly 10 or 100 of filters, we will have a large amount of parameters.
- We can use pooling layers to reduce this.
- Pooling layers accept convolutional layers as input: Input → Conv → Pool.



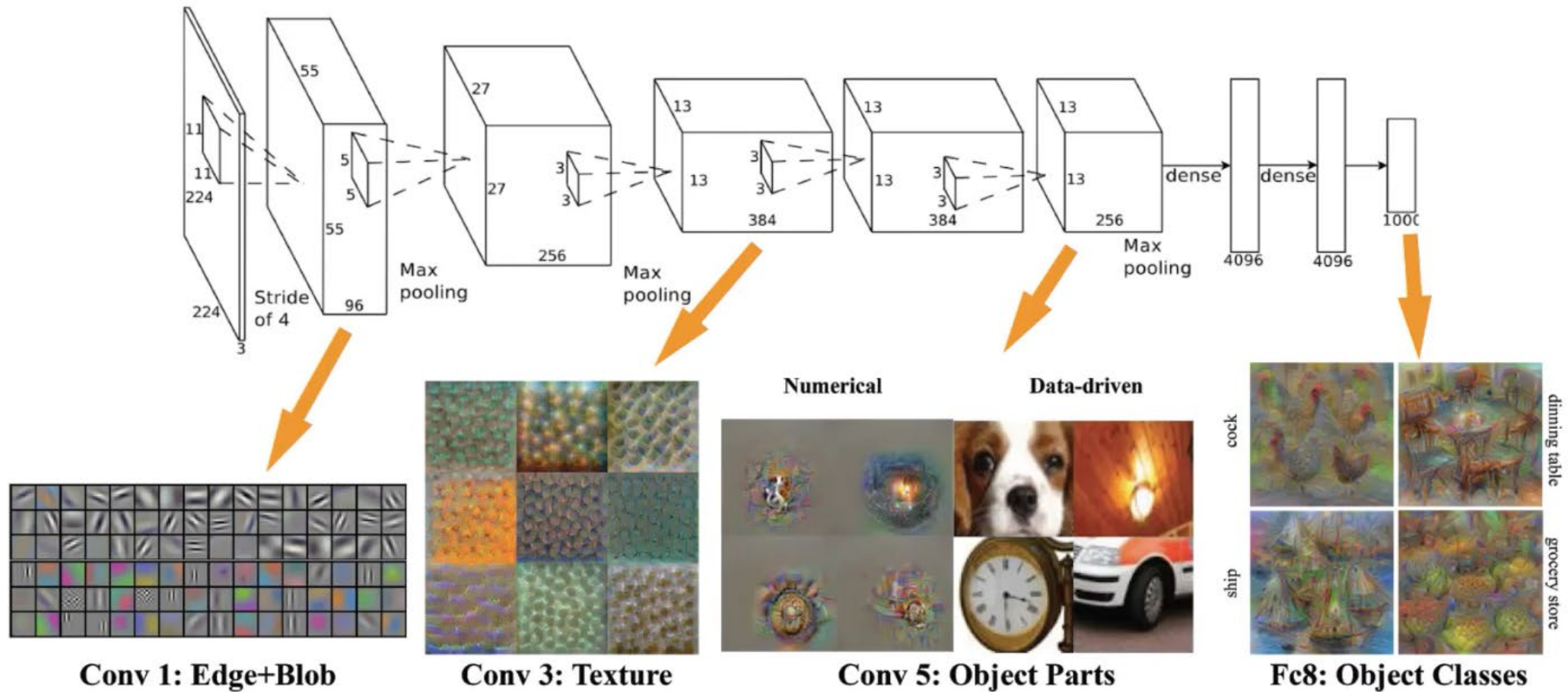
Dropout Layer

- Common technique to help out with the training time and overfitting problems → dropout layer.
- During training, units are randomly dropped, along with their connections.
- Meaning that we provide the percentage (ex:50%) that is going to be turned off when training process.

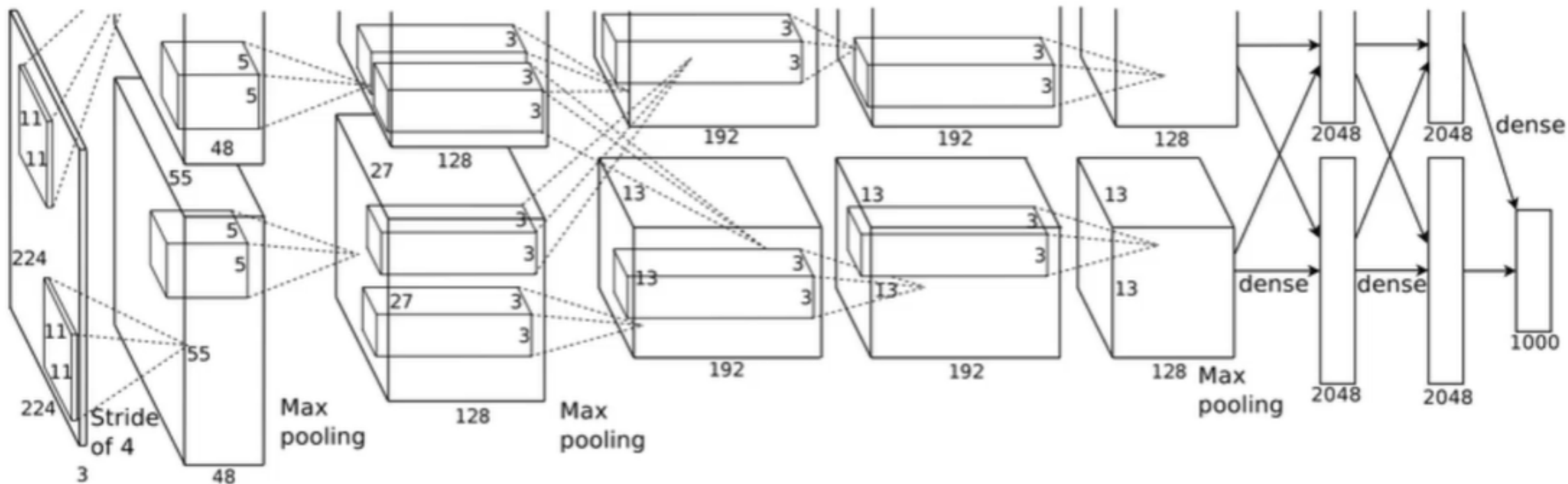
Architecture of CNN

- LeNet-5 by Yann Lecun.
- AlexNet by Alex Krizhevsky et al.
- GoogLeNet by Szegedy at Google Research.
- ResNet by Kaiming He et al.
- Etc.

AlexNet (1)

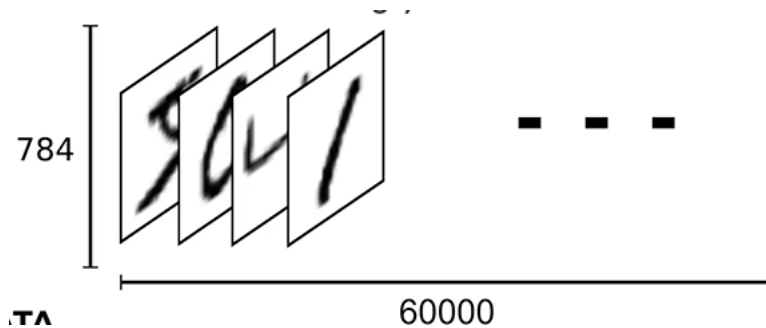


AlexNet (2)



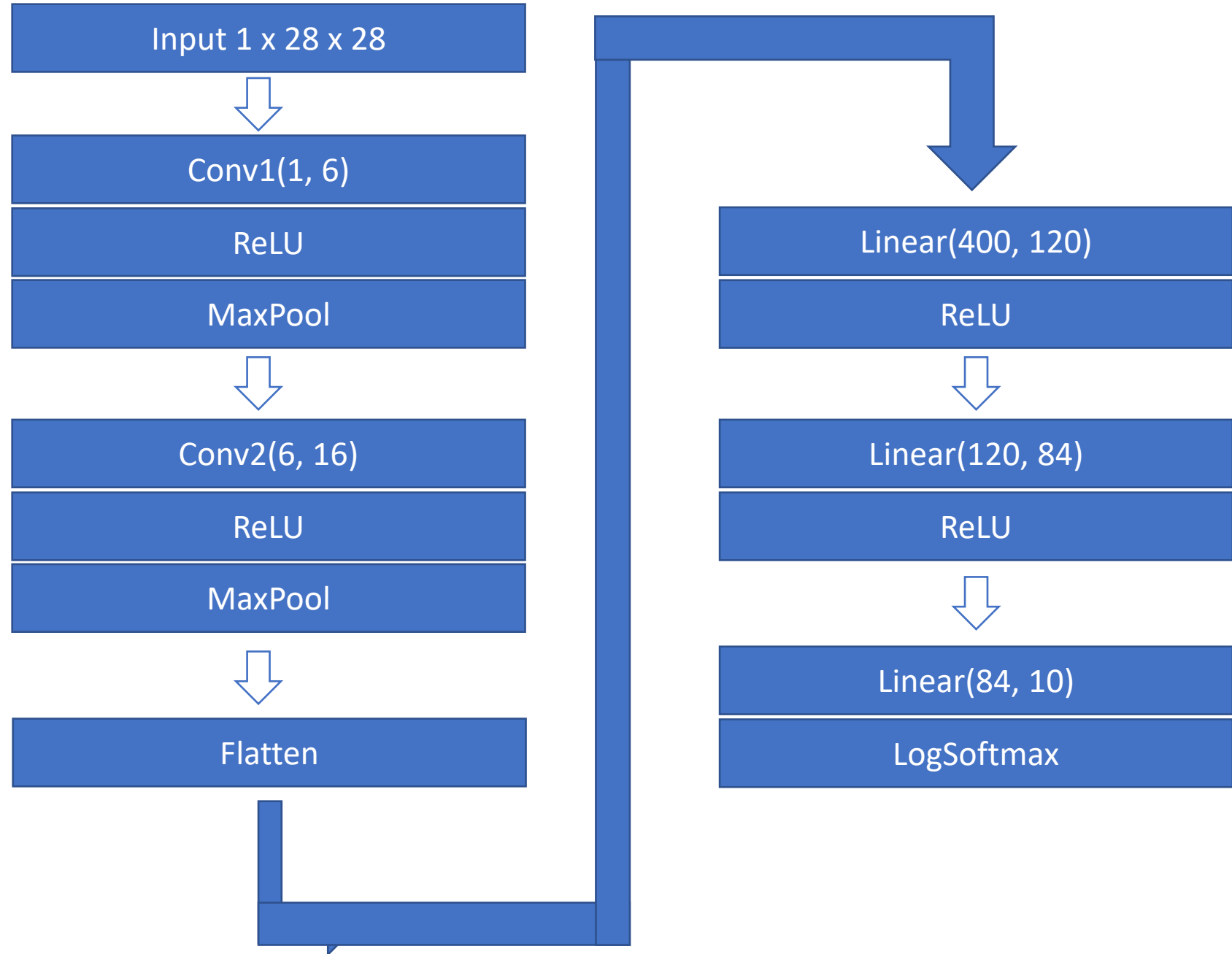
MNIST using CNN

- We can think of the entire group of the 60,000 images as a tensor (an n-dimensional array).



- For the labels we will use One-Hot Encoding → labeling such as “One”, “Two”, etc... we’ll have a single array for each image.
- The corresponding label will be a 1 at the index location and zero everywhere else. Ex: 4 would have label array = [0,0,0,0,1,0,0,0,0,0]
- When dealing with tensors of image data, we actually end up with 4 dimensions : Number of images, height, width, color channels.

Architecture



Import Library

```
import torch
import torch.nn as nn
import torch.nn.functional as F # adds some efficiency
```

```
from torch.utils.data import DataLoader # Lets us load data in batches
from torchvision import datasets, transforms
from torchvision.utils import make_grid
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix # for evaluating results
import matplotlib.pyplot as plt
%matplotlib inline
```

1. Load MNIST Dataset

```
#PyTorch menyediakan dataset MNIST melalui torchvision  
#Pada saat awal dipanggil, dataset didownload ke path tertentu di PC
```

```
#Pertama kita ubah image MNIST ke tensor  
transform = transforms.ToTensor()
```

```
#download dataset ke path tertentu  
#input 2 variabel ==> data training dan data testing  
  
train_data = datasets.MNIST(root='../CNN\Data', train=True, download=True, transform=transform)  
test_data = datasets.MNIST(root='../CNN\Data', train=False, download=True, transform=transform)
```

```
train_data
```

```
Dataset MNIST  
  Number of datapoints: 60000  
  Root location: ../CNN\Data  
  Split: Train  
  StandardTransform  
  Transform: ToTensor()
```

```
test_data
```

```
Dataset MNIST  
  Number of datapoints: 10000  
  Root location: ../CNN\Data  
  Split: Test  
  StandardTransform  
  Transform: ToTensor()
```

2. View Image

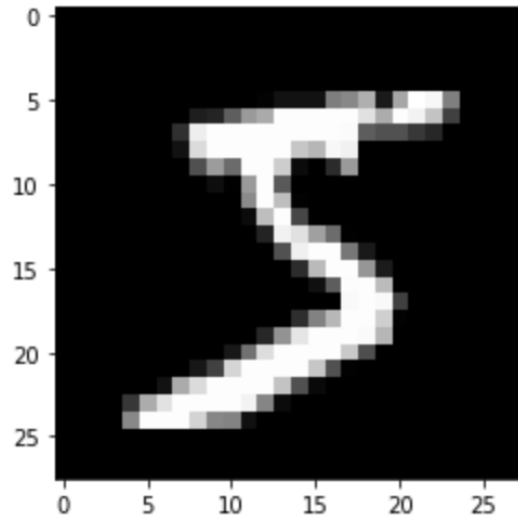
```
#cek apakah huruf data train ke 0?  
#cek labelling, disini dataset MNIST sudah memiliki label
```

```
image, label = train_data[0]  
print('Shape:', image.shape, '\nLabel:', label)
```

```
Shape: torch.Size([1, 28, 28])  
Label: 5
```

```
#cek Label 5 itu huruf apa? ditampilkan gambarnya
```

```
plt.imshow(image.reshape(28,28), cmap="gray");
```



3. Batch loading with DataLoader

```
#Data train ada 60.000 => sangat banyak parameter yang diperlukan untuk arsitektur  
#Agar efisien, dilakukan training kecil2 menggunakan DataLoader  
#Pada data citra, biasanya digunakan small batch  
#Kita coba mulai dari batch kecil lalu batch besar  
  
train_loader = DataLoader(train_data, batch_size=10, shuffle=True)  
test_loader = DataLoader(test_data, batch_size=10, shuffle=False) #data test tidak perlu di acak / shuffle
```

4. Define Convolutional Model (1)

```
# Define Layers

# panggil konvolusi 2D, karena citra
#setelah ketik nn.Conv2d klik Shift+Tab untuk mengetahui menu isian

#input channel = 1 karena citranya grayscale, output channels = 6 feature maps atau 6 filter, kernel size = 3x3, stride = 1
conv1 = nn.Conv2d(1, 6, 3, 1) # --> 6 filters --> pooling --> conv2
#input channel = 6 mengikuti output conv1, output channel = 16 (kalo bisa 2^n), kernel size = 3x3, stride =1
conv2 = nn.Conv2d(6, 16, 3, 1)

# Grab the first MNIST record

for i, (X_train, y_train) in enumerate(train_data):
    break
```

- train_data is consisted of image data (X_train) and the label (y_train).
- Since there is 60,000 images and the batch size = 10, then the number of looping = $60,000 / 10 = 6000$ times → so if we want to show just 1 loop as first batch, add “break” after looping.

4. Define Convolutional Model (2)

```
#cek ukuran single x_train  
X_train.shape
```

```
torch.Size([1, 28, 28])
```

```
#y_train = target = label  
#cek y_train pertama  
y_train
```

```
5
```

```
#kita sudah desain untuk data train ada 10 batch_size ==> ada 10 citra setiap batch, data 4D  
#4D tensor  
#1 batch, 1 color channel image, 28 x 28 shape  
x = X_train.view(1,1,28,28) # --> 4D batch (batch of 1 image)  
x
```

```
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.9922, 0.9922, 0.9922, 0.9569, 0.5216, 0.0431, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000],  
[0.0000, 0.0000, 0.0000, 0.0000, 0.5333, 0.9922, 0.9922, 0.9922,  
 0.8314, 0.5294, 0.5176, 0.0627, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000],  
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000],  
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000],  
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,  
 0.0000, 0.0000, 0.0000, 0.0000]]])
```

- Because we splitted train_data into X_train and y_train, so if we see X_train, the label disappear

4. Define Convolutional Model (3): First conv

```
# Perform the first convolution/activation from single image x  
# choose fungsi aktivasi rectified linier unit (relu)
```

```
x = F.relu(conv1(x))  
print(x.shape)
```

```
torch.Size([1, 6, 26, 26])
```

```
# Artinya apa dari hasil shape 26x26 ini? Bukankah tadi citranya berukuran 28x28?  
# Efek dari konvolusi --> ukuran citra semakin kecil --> mengurangi informasi citra di bagian border  
# Seharusnya ditambah padding untuk menjaga ukuran citra, namun mengapa tidak ditambahkan padding disini?  
# Kita harus case by case. Karena citra MNIST bordernya tidak mengandung informasi, maka tidak perlu kita tambah padding  
  
# torch.Size([1, 6, 26, 26]) --> 1 image, 6 filters, image size 26x26
```

```
# Run the first pooling layer
```

```
# input = x, kernel size = 2x2, stride 2  
# Mengapa stride = 2? karena kernel size nya 2x2 --> pelajari lagi konsep pooling layer
```

```
x = F.max_pool2d(x, 2, 2)  
print(x.shape)
```

```
torch.Size([1, 6, 13, 13])
```

```
# hasil nya berukuran 13x13 karena dilakukan kernel size 2x2 pada pooling layer  
# menghasilkan citra berukuran separuhnya  
# Hasil akhir dari layer 1
```

```
#####
```


4. Define Convolutional Model (4): Second conv

```
# Perform the second convolution/activation  
x = F.relu(conv2(x))  
print(x.shape)
```

```
torch.Size([1, 16, 11, 11])
```

```
# 1 input, 16 filters, image size = 11x11 akibat proses konvolusi  
# image size hasil pooling layer 1 = 13x13, skrg berkurang menjadi 11x11
```

```
# Run the second pooling layer  
x = F.max_pool2d(x, 2, 2)  
print(x.shape)
```

```
torch.Size([1, 16, 5, 5])
```

4. Define Convolutional Model (5): Last conv

```
# Flatten the data  
#-1 assumes whatever first batch size, we won't be dealing with just one single image  
#keep the first dimension  
x = x.view(-1, 5*5*16)  
print(x.shape)
```

```
torch.Size([1, 400])
```

```
# satu citra ada data flatten = 400  
# jika ada 10 citra MNIST yg mewakili angka 0-9 maka ada 10x400
```

5. Architecture: Define Convolutional Model

```
class ConvolutionalNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 3, 1) #1color channel, 6 filters (output channel from layer1), 3x3 kernel, stride=1
        self.conv2 = nn.Conv2d(6, 16, 3, 1)
        self.fc1 = nn.Linear(5*5*16, 120) #for fully connected layer, dgn 120 neurons
        self.fc2 = nn.Linear(120, 84) #120 input dari node fc1, jumlah neuron berikutnya dikurangi
        self.fc3 = nn.Linear(84,10) #output layer dari fully connected

    def forward(self, X):
        X = F.relu(self.conv1(X))
        X = F.max_pool2d(X, 2, 2) #input=X, kernel=2x2, stride=2
        X = F.relu(self.conv2(X))
        X = F.max_pool2d(X, 2, 2)
        X = X.view(-1, 5*5*16) #-1 agar kita bisa memvariasikan batch size
        X = F.relu(self.fc1(X))
        X = F.relu(self.fc2(X))
        X = self.fc3(X)
        return F.log_softmax(X, dim=1) #fungsi aktivasi bagian akhir, dimension=1 karena sudah 1 array flatten
```

```
torch.manual_seed(42) #42 = set weight tertentu yang ditetapkan
model = ConvolutionalNetwork()
model
```

```
ConvolutionalNetwork(
  (conv1): Conv2d(1, 6, kernel_size=(3, 3), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(3, 3), stride=(1, 1))
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

Total number of parameters

- Including the bias terms for each layer, the total number of parameters being trained is:

$$(1 \times 6 \times 3 \times 3) + 6 + (6 \times 16 \times 3 \times 3) + 16 + (400 \times 120) + 120 + (120 \times 84) + 84 + (84 \times 10) + 10 =$$
$$54 + 6 + 864 + 16 + 48000 + 120 + 10080 + 84 + 840 + 10 = 60,074$$

```
for param in model.parameters():  
    print(param.numel()) #numel = number of elements / parameters
```

```
54  
6  
864  
16  
48000  
120  
10080  
84  
840  
10
```

6. Define loss function & optimizer

```
criterion = nn.CrossEntropyLoss() #karena multiclass classification  
optimizer = torch.optim.Adam(model.parameters(), lr=0.001) #Learning rate  
  
#jika learning rate terlalu kecil, maka optimizer akan butuh iterasi lama  
#jika lr terlalu besar, beresiko akurasi kurang ==> overfitting, atau malah tidak konvergen
```

7. Train the model (1)

```
import time
start_time = time.time()

#VARIABLES
epochs = 5
train_losses = []
test_losses = []
train_correct = []
test_correct = []

for i in range(epochs):
    trn_corr = 0 #training correct
    tst_corr = 0 #test correct

    # TRAIN batches, b = batches
    for b, (X_train, y_train) in enumerate(train_loader):
        b+=1

        # Apply the model
        y_pred = model(X_train) # we don't flatten X-train here, keep data in 2D image
        loss = criterion(y_pred, y_train) #cek selisih error

        # The number of correct predictions
        predicted = torch.max(y_pred.data, 1)[1] #check print(y_pred.data) to know data of one epoch, 1 = actual predicted value
        batch_corr = (predicted == y_train).sum() #berapa yg correct dari batch dimana predicted points = train points ==> true 1
        trn_corr += batch_corr #jumlah correct selama training

        # Update parameters
        optimizer.zero_grad() # Clear gradients ==> parameters
        loss.backward() # Getting gradients ==> parameters
        optimizer.step() # Updating parameters

        # Print interim results, cek misalnya saat batch=600
        if b%600 == 0:
            print(f'epoch: {i}  batch: {b}  loss: {loss.item()}')

    train_losses.append(loss.item())
    train_correct.append(trn_corr.item())
```

7. Test the model (2)

```
# TESTING batches
with torch.no_grad():
    for b, (X_test, y_test) in enumerate(test_loader):

        # Apply the model
        y_val = model(X_test)

        # calculate the number of correct predictions
        predicted = torch.max(y_val.data, 1)[1]
        tst_corr += (predicted == y_test).sum()

    loss = criterion(y_val, y_test)
    test_losses.append(loss)
    test_correct.append(tst_corr)

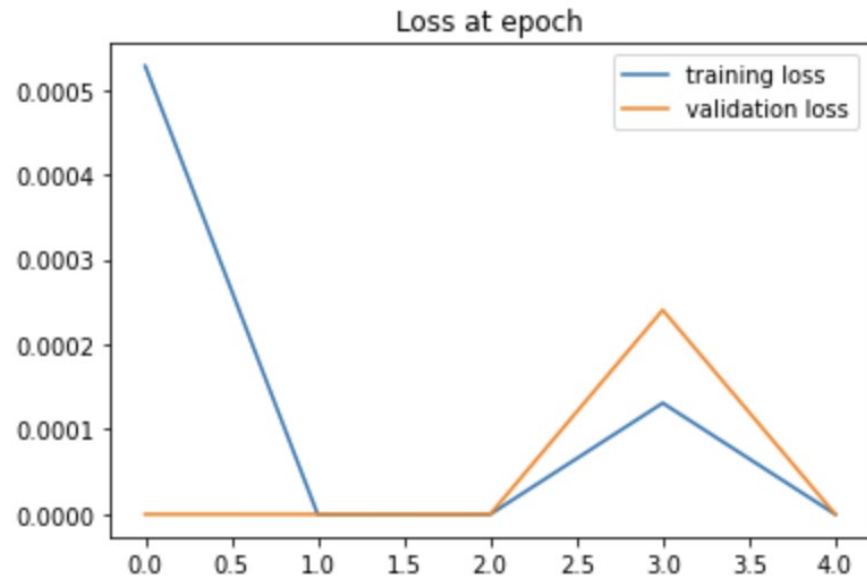
    current_time = time.time()
    total = current_time - start_time
print(f'\nDuration: {total/60} minutes') # print the time elapsed
```

```
epoch: 0 batch: 600 loss: 0.040556252002716064
epoch: 0 batch: 1200 loss: 0.08253474533557892
epoch: 0 batch: 1800 loss: 0.36018985509872437
epoch: 0 batch: 2400 loss: 0.01818694919347763
epoch: 0 batch: 3000 loss: 0.00846516527235508
epoch: 0 batch: 3600 loss: 0.001142998575232923
epoch: 0 batch: 4200 loss: 0.624707818031311
epoch: 0 batch: 4800 loss: 0.046218063682317734
epoch: 0 batch: 5400 loss: 0.00839207973331213
epoch: 0 batch: 6000 loss: 0.044792745262384415
epoch: 1 batch: 600 loss: 0.003464172128587961
epoch: 1 batch: 1200 loss: 0.048956435173749924
epoch: 1 batch: 1800 loss: 0.000778083922341466
epoch: 1 batch: 2400 loss: 0.04166005551815033
epoch: 1 batch: 3000 loss: 0.19562456011772156
epoch: 1 batch: 3600 loss: 0.002942086197435856
epoch: 1 batch: 4200 loss: 0.0005433653714135289
epoch: 1 batch: 4800 loss: 0.0005624962504953146
epoch: 1 batch: 5400 loss: 0.00030035508098080754
epoch: 1 batch: 6000 loss: 0.03586563095450401
epoch: 2 batch: 600 loss: 0.002943187253549695
epoch: 2 batch: 1200 loss: 0.0014550242340192199
epoch: 2 batch: 1800 loss: 0.001368862111121416
epoch: 2 batch: 2400 loss: 0.005534386727958918
epoch: 2 batch: 3000 loss: 0.4764808118343353
epoch: 2 batch: 3600 loss: 0.05505165457725525
epoch: 2 batch: 4200 loss: 0.007271642331033945
epoch: 2 batch: 4800 loss: 0.005500622093677521
epoch: 2 batch: 5400 loss: 0.00019268484902568161
epoch: 2 batch: 6000 loss: 7.886278035584837e-05
epoch: 3 batch: 600 loss: 0.0005187172791920602
epoch: 3 batch: 1200 loss: 0.00213501020334661
epoch: 3 batch: 1800 loss: 5.13993545609992e-05
epoch: 3 batch: 2400 loss: 0.00013268116163089871
epoch: 3 batch: 3000 loss: 0.0001986158749787137
epoch: 3 batch: 3600 loss: 0.002073561307042837
epoch: 3 batch: 4200 loss: 0.00023550578043796122
epoch: 3 batch: 4800 loss: 0.07387697696685791
epoch: 3 batch: 5400 loss: 0.008837352506816387
epoch: 3 batch: 6000 loss: 0.03450579196214676
epoch: 4 batch: 600 loss: 0.001475363620556891
epoch: 4 batch: 1200 loss: 0.07949581742286682
epoch: 4 batch: 1800 loss: 2.6140929549001157e-05
epoch: 4 batch: 2400 loss: 0.0003961739712394774
epoch: 4 batch: 3000 loss: 0.03030974790453911
epoch: 4 batch: 3600 loss: 0.013598117046058178
epoch: 4 batch: 4200 loss: 0.0017820881912484765
epoch: 4 batch: 4800 loss: 0.0009464075556024909
epoch: 4 batch: 5400 loss: 6.844296149211004e-05
epoch: 4 batch: 6000 loss: 0.0001983713882509619
```

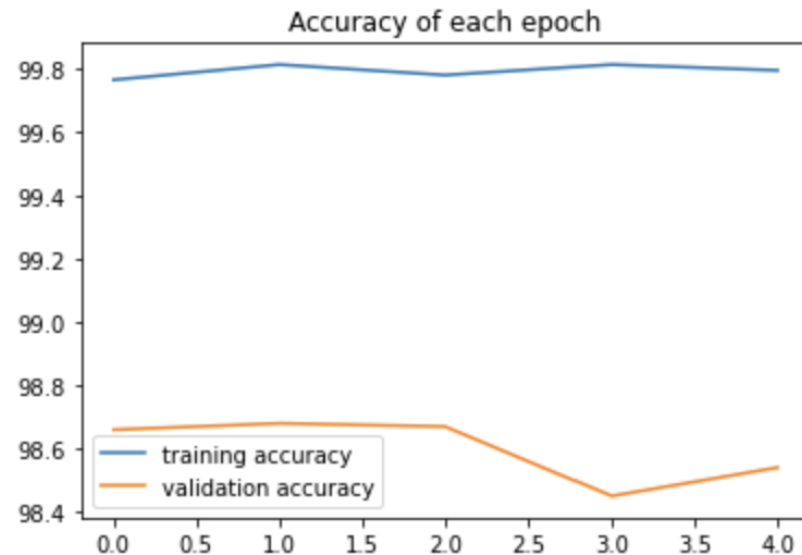
Duration: 1.9850836873054505 minutes

8. The loss and accuracy comparisons

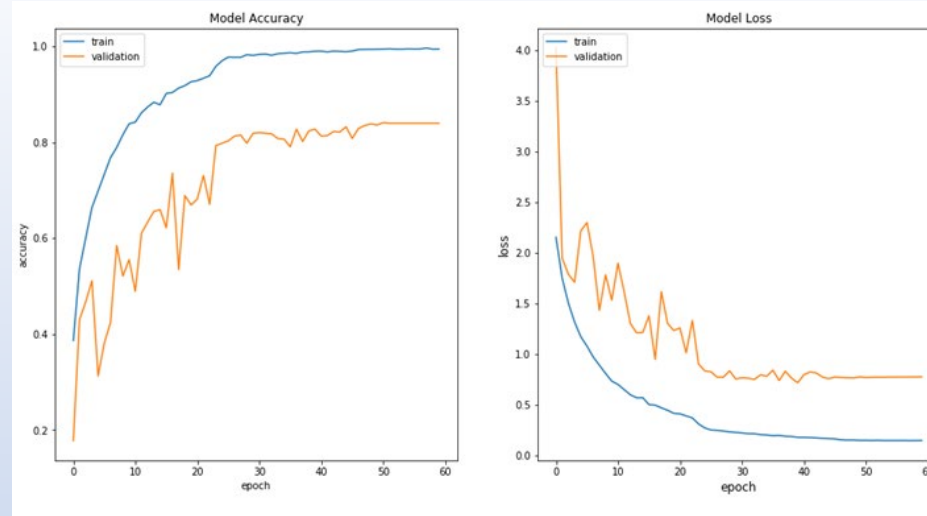
```
plt.plot(train_losses, label='training loss')
plt.plot(test_losses, label='validation loss')
plt.title('Loss at epoch')
plt.legend();
```



```
#accuracy at each epoch
plt.plot([t/600 for t in train_correct], label='training accuracy')
plt.plot([t/100 for t in test_correct], label='validation accuracy')
plt.title('Accuracy of each epoch')
plt.legend();
```



Accuracy and Loss



- Accuracy is a method for measuring a classification model's performance. It is typically expressed as a percentage. Accuracy is the count of predictions where the predicted value is equal to the true value. It is binary (true/false) for a particular sample. Accuracy is often graphed and monitored during the training phase though the value is often associated with the overall or final model accuracy. Accuracy is easier to interpret than loss.
- A loss function, also known as a cost function, takes into account the probabilities or uncertainty of a prediction based on how much the prediction varies from the true value. This gives us a more nuanced view into how well the model is performing.
- Unlike accuracy, loss is not a percentage — it is a summation of the errors made for each sample in training or validation sets. Loss is often used in the training process to find the "best" parameter values for the model (e.g. weights in neural network). During the training process the goal is to minimize this value.
- The most common loss functions are **log loss** and **cross-entropy loss** (which yield the same result when calculating error rates between 0 and 1), as well as **mean squared error**, and **likelihood loss**.
- Unlike accuracy, loss may be used in both classification and regression problems.

Evaluating all the test data

```
# Evaluating all the test data
# Extract the data all at once, not in batches
test_load_all = DataLoader(test_data, batch_size=10000, shuffle=False)
# if we want to train the data, do shuffle = True, if not, just test, do shuffle = false

with torch.no_grad(): #don't update weight and bias in test data
    correct = 0
    for X_test, y_test in test_load_all:
        y_val = model(X_test) # we don't flatten the data this time
        predicted = torch.max(y_val,1)[1]
        correct += (predicted == y_test).sum()
print(f'Test accuracy: {correct.item()}/{len(test_data)} = {correct.item()*100/(len(test_data))}%')
```

Test accuracy: 9854/10000 = 98.54%

- The accuracy is higher than ANN (97.65%) with 10 epoch. However, in this CNN, we just set 5 epoch.

9. Confusion Matrix

```
# print a row of values for reference
np.set_printoptions(formatter=dict(int=lambda x: f'{x:4}')) #x:4 untuk memberikan spasi antar data
print(np.arange(10).reshape(1,10))
print()

# print the confusion matrix
print(confusion_matrix(predicted.view(-1), y_test.view(-1)))
```

```
[[ 0  1  2  3  4  5  6  7  8  9]]

[[ 972  0  2  0  0  2  3  0  2  0]
 [  0 1135  4  1  1  0  4  7  0  3]
 [  2  0 1014  0  0  0  1  3  1  1]
 [  0  0  0 1000  0 17  0  0  1  0]
 [  0  0  1  0 979  0  2  0  1 16]
 [  0  0  0  1  0 859  0  0  1  2]
 [  3  0  0  0  0  3 945  0  0  0]
 [  1  0 11  5  0  1  0 1016  1  5]
 [  1  0  0  2  0  4  3  1 965 11]
 [  1  0  0  1  2  6  0  1  2 971]]
```

How to test single image? (1)

#Coba Tes 1 citra

```
test_data[2019]
```

[illegible]

9)

```
#terlihat pada citra di atas berlabel 9
```

```
#kita cek hasil citradari data tes tsb
```

#pertama kita hilangkan labelnya

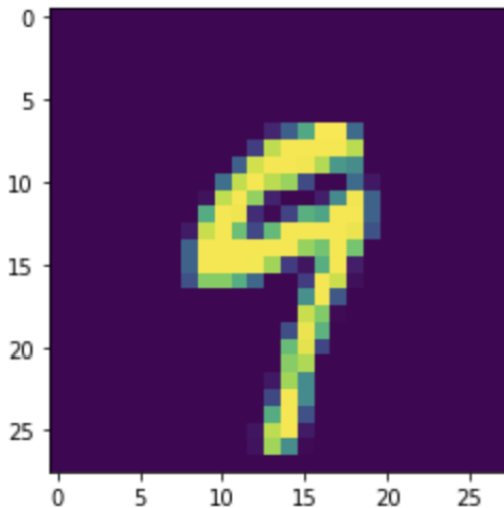
```
test_data[2019][0]
```

[illegible]

How to test single image? (2)

```
plt.imshow(test_data[2019][0].reshape(28,28)) #coba cek dengan tes data array lain  
# ada 10000 data tes, sehingga kita bisa melakukan pengujian dari array 0..9999
```

<matplotlib.image.AxesImage at 0x236aa900520>



#coba prediksi citra tersebut

```
model.eval()
```

```
with torch.no_grad():
```

```
    new_pred = model(test_data[2019][0].view(1,1,28,28)).argmax() #1 bath size, 1 color channel
```

```
print("Predicted value:",new_pred.item())
```

#argmax untuk mengambil nilai probabilitas max. Setiap prediksi 0 - 9 akan dihitung probabilitasnya

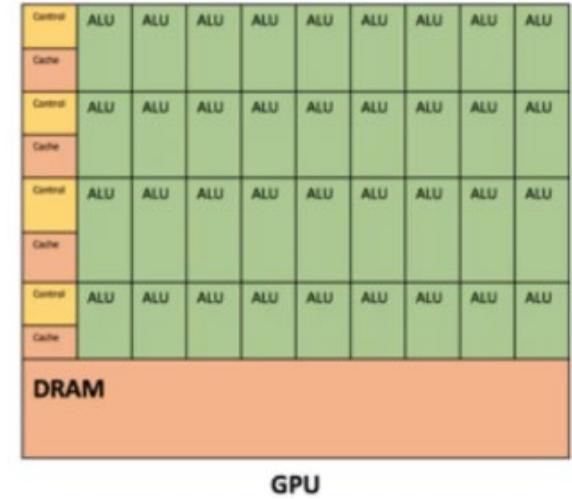
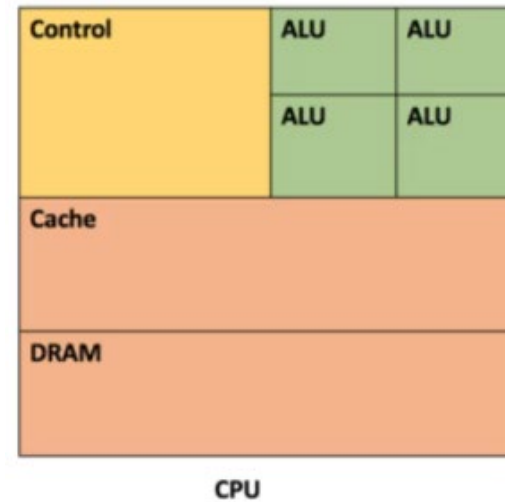
Predicted value: 9

new_prediction

```
tensor([[ -52.1155, -64.9820, -64.8526, -52.4779, -43.5864, -46.9069, -69.7649,  
         -46.4392, -42.4981,   0.0000]])
```

Using GPU for PyTorch

Why do we need GPU? (1)

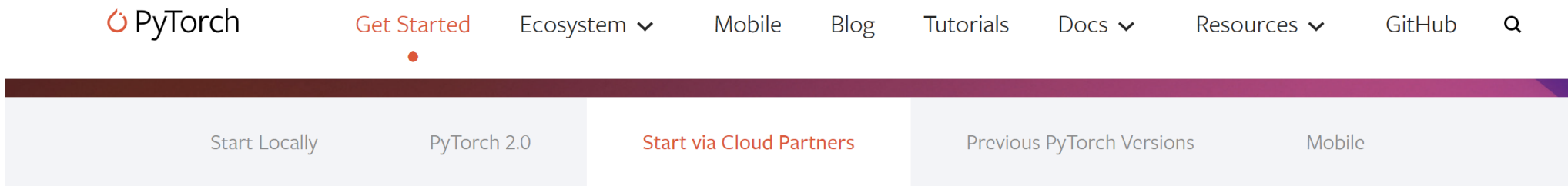


- Typically for larger networks dealing with large datasets, training on a GPU will be much faster than training on a CPU.
- Note: **Not every computer will have suitable GPU for PyTorch.**
- The backpropagation process is dependent on matrix multiplication, for which GPUs are much faster.
- But why are GPUs better at this than a CPU?
- Fundamentally, GPUs have many more logical cores (Arithmetic Logic Units, ALUs).

Why do we need GPU? (2)

- NVIDIA created a parallel computing architecture known as CUDA, which gave deep learning library developers ability to write code for parallel processing on a GPU.
- This allows for GPUs to have much larger bandwidths for processing information.
- CUDA is currently limited to NVIDIA hardware.
- PyTorch uses CUDA for its GPU acceleration → This means that GPU with PyTorch will require to have an NVIDIA GPU card.

Don't have NVIDIA hardware?



START VIA CLOUD PARTNERS

Cloud platforms provide powerful hardware and infrastructure for training and deploying deep learning models. Select a cloud platform below to get started with PyTorch.

	Amazon Web Services	>
	Google Cloud Platform	>
	Microsoft Azure	>

Installation (1)

- Download and install NVIDIA CUDA Toolkit :
<https://developer.nvidia.com/cuda-downloads>
- After downloading, the version of PyTorch will be changed into new version, so that we have to install PyTorch again ➡ it is recommended install in new environment.
- To create an environment with a specific version of Python:
- `conda create -n myenv python=3.9`

Installation (2)

- Check the version of Pythorch available in [Start Locally | PyTorch](#)

PyTorch Build	Stable (2.0.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.7	CUDA 11.8	ROCm 5.4.2	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu117</pre>			

- Then, download CUDA Toolkit at [CUDA Toolkit 11.7 Downloads | NVIDIA Developer](#). The version of CUDA Toolkit must as same as CUDA that available in PyTorch. Install CUDA Toolkit.

CUDA Toolkit 11.7 Downloads

Home

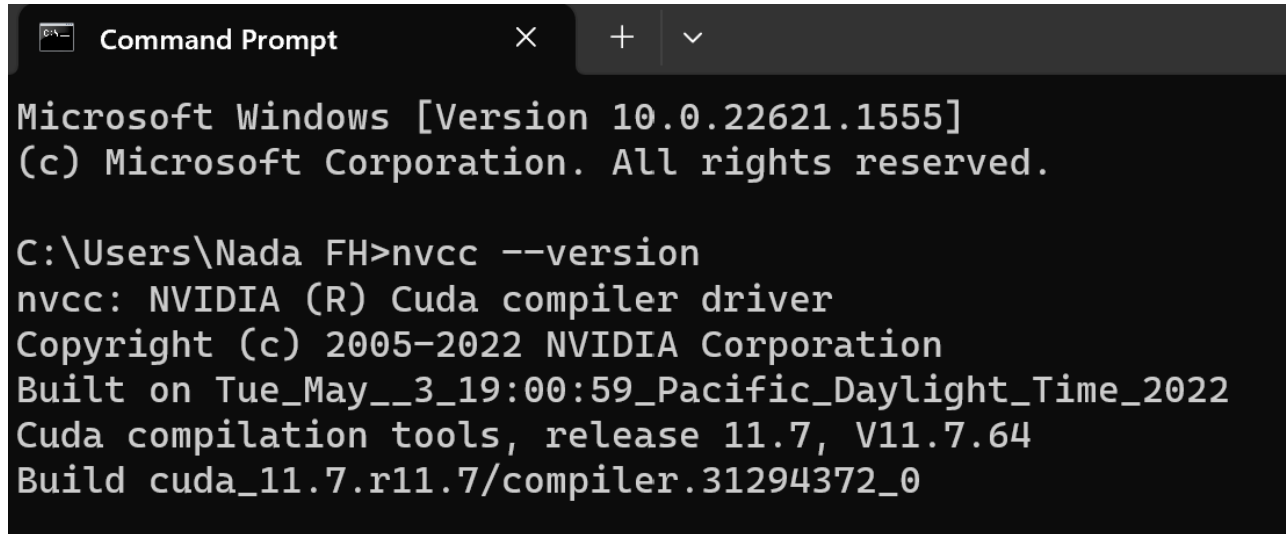
Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux	Windows			
Architecture	x86_64				
Version	10	11	Server 2016	Server 2019	Server 2022
Installer Type	exe (local)	exe (network)			

Installation (3)

- Check CUDA is installed or not: open command prompt →
- type “nvcc --version”



```
Command Prompt
Microsoft Windows [Version 10.0.22621.1555]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nada FH>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Tue_May__3_19:00:59_Pacific_Daylight_Time_2022
Cuda compilation tools, release 11.7, V11.7.64
Build cuda_11.7.r11.7/compiler.31294372_0
```

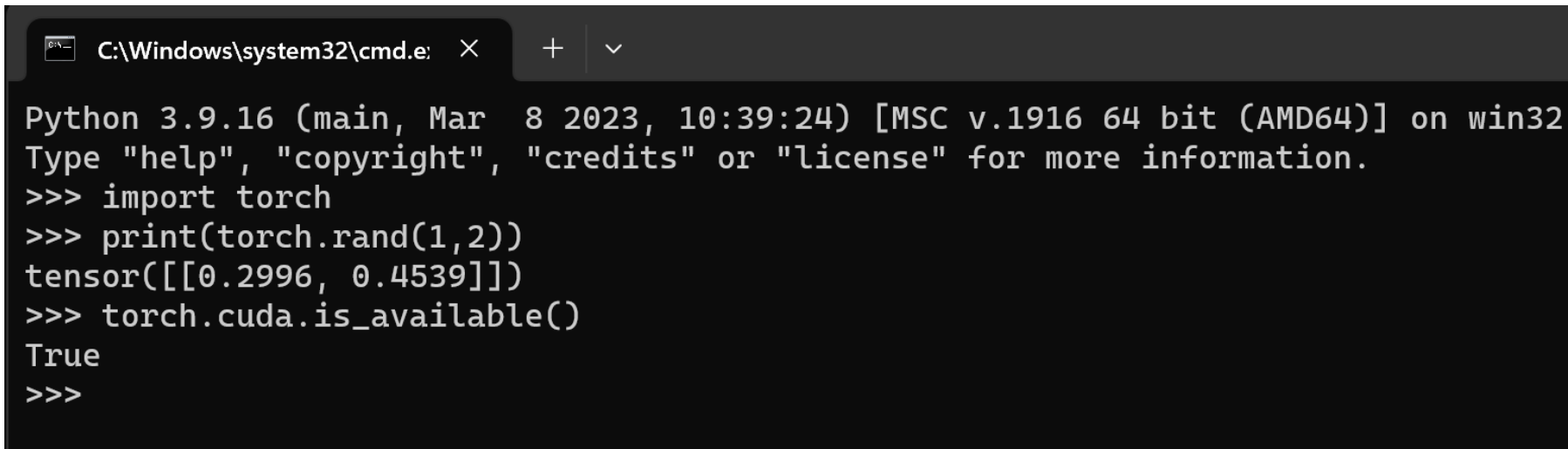
Installation (4)

- Copy paste the command in conda prompt, don't forget to install in new environment.

PyTorch Build	Stable (2.0.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.7	CUDA 11.8	ROCm 5.4.2	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu117</pre>			

Installation (5)

- Verify whether torch and cuda is installed successfully or not → open anaconda → Environments → select the environment → click open with python.

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\system32\cmd.e' and standard window controls. The window contains a Python 3.9.16 shell session. The user has imported torch, printed a random tensor, and checked if CUDA is available, which returned True.

```
C:\Windows\system32\cmd.e  X  +  v

Python 3.9.16 (main, Mar 8 2023, 10:39:24) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.rand(1,2))
tensor([[0.2996, 0.4539]])
>>> torch.cuda.is_available()
True
>>>
```

Installation (6)

- In new env, type : conda install pip, conda install ipykernel.
- Add new kernel in jupyter notebook, open anaconda prompt, make sure you're in the new env, then type:
- `python -m ipykernel install --user --name newcuda3 --display-name "Python (newcuda3)"`
- In the highlighted is the example of my environment.
- Verify CUDA is available in notebook, firstly select kernel into "Python (newcuda3)", then type:

```
import torch
torch.cuda.is_available()
```

True

Using GPU and CUDA

- Not every computer can run GPU, we need CUDA and an NVIDIA GPU.
- Check ID and name of GPU device:

```
## Get Id of default device  
torch.cuda.current_device()
```

0

```
# 0  
torch.cuda.get_device_name(0) # Get name device with ID '0'
```

'NVIDIA GeForce RTX 3050 Laptop GPU'

- Check how much memory allocated on GPU. The result is 0, meaning that this computer don't use memory for CUDA previously.

```
# Check memory  
torch.cuda.memory_allocated()
```

0

Compare CPU and GPU

```
# CPU  
a = torch.FloatTensor([1., 2.])
```

```
a  
  
tensor([1., 2.])
```

```
a.device  
  
device(type='cpu')
```

```
# GPU  
a = torch.FloatTensor([1., 2.]).cuda()
```

```
a.device  
  
device(type='cuda', index=0)
```

```
torch.cuda.memory_allocated()  
  
512
```

- 512 bytes being allocated

Convert Tensor to CUDA Tensor

```
X_train = torch.FloatTensor(X_train).cuda()  
X_test = torch.FloatTensor(X_test).cuda()  
y_train = torch.LongTensor(y_train).cuda()  
y_test = torch.LongTensor(y_test).cuda()
```